

STAFF ENGINEER STUDY GUIDE

Microservices Patterns

A simple, diagram-driven summary of the key ideas

Distilled from “*Microservices Patterns — With examples in Java*” by Chris Richardson (Manning, 2019). Plain-language explanations, concrete FTGO examples, pattern cards, and visual diagrams for all 13 chapters.

Contents

- 1 Escaping monolithic hell**
Monolith pain, the Scale Cube, what microservices are, benefits & drawbacks
- 2 Decomposition strategies**
Software architecture, decompose by business capability & by subdomain (DDD)
- 3 Interprocess communication**
REST/gRPC, circuit breaker, service discovery, messaging, transactional outbox
- 4 Managing transactions with sagas**
Why 2PC is avoided, sagas, choreography vs orchestration, isolation anomalies
- 5 Designing business logic**
Transaction script vs domain model, DDD aggregates, domain events
- 6 Event sourcing**
Storing events not state, event store, benefits & challenges
- 7 Implementing queries**
API composition and CQRS read models
- 8 External API patterns**
API gateway and Backends for Frontends (BFF)
- 9 Testing microservices — Part 1**
Test pyramid, test doubles, consumer-driven contract tests
- 10 Testing microservices — Part 2**
Integration, component, and end-to-end testing
- 11 Production-ready services**
Security/access tokens, config, observability, chassis & service mesh
- 12 Deploying microservices**
JAR → VM → container → Kubernetes → serverless; sidecar & service mesh
- 13 Refactoring to microservices**
Strangler application & anti-corruption layer

This chapter asks: why do teams that started fast with a single, simple codebase end up stuck, shipping slower every quarter? It uses FTGO, a fictional food-delivery app (think DoorDash/Uber Eats), to show how a monolith rots, then introduces the "Scale Cube" as a mental model for the three ways you can scale a system, and lands on a working definition of microservice architecture. For a staff engineer, this chapter is the "why" you'll need to justify any re-architecture to skeptical leadership.

The FTGO example

FTGO connects three kinds of users: **Consumers** who order food, **Restaurants** who prepare it, and **Couriers** who deliver it. Underneath, the app has to handle orders, kitchen tickets, delivery assignment, payments, and accounting. Richardson uses this one example for the whole book, so it's worth remembering the cast: Order, Kitchen, Delivery, Restaurant, Consumer, Accounting.

FTGO started as a classic **monolith**: one codebase, one build, one deployable artifact (e.g. one WAR/JAR, or one container image), one database, running on a cluster of identical instances behind a load balancer.

BENEFITS

A monolith is genuinely a good choice when a product is small or young: it's simple to develop (one IDE, one repo), simple to test end-to-end, simple to deploy (copy one artifact), and simple to scale horizontally by running more copies. Most successful startups, including the fictional FTGO, begin this way for good reason.

Monolithic hell

The problem shows up later, as the app and the team grow. Richardson calls this **monolithic hell**: the codebase gets so large and tangled that no single engineer understands all of it, the IDE and build get slow, every change risks breaking something unrelated, and the whole app must be redeployed for even a one-line fix. Three symptoms matter most:

TRADEOFFS / WATCH OUT — SYMPTOMS OF MONOLITHIC HELL

Slow, risky delivery. A big shared codebase means big shared risk. Every deploy ships everyone's changes at once, so releases need heavyweight coordination, manual regression testing, and off-hours "deploy windows." Velocity drops as the codebase grows.

Hard to keep reliable. One buggy module (say, a memory leak in the analytics code) can crash the whole process and take payments and ordering down with it. There's no fault isolation — everything shares one runtime.

Stuck on an old tech stack. Because everything is one codebase compiled and deployed as one unit, you can't adopt a new language, framework, or database for just one part of the system without a full, risky, all-or-nothing rewrite. "Let a thousand flowers bloom" isn't possible.

Analogy: a monolith is like one massive shared Google Doc that the whole company edits. Early on, that's fine — fast, simple, everyone sees everything. At 500 people editing simultaneously, you get merge

conflicts, accidental deletions, and nobody dares touch the "scary" section. Splitting it into separate docs per team (microservices) trades a little coordination overhead for the ability to work independently.

The Scale Cube

Before defining microservices, the book borrows a model from *The Art of Scalability* called the **Scale Cube**: three independent ways to scale an application. Any real system usually combines all three, but each axis solves a different problem.

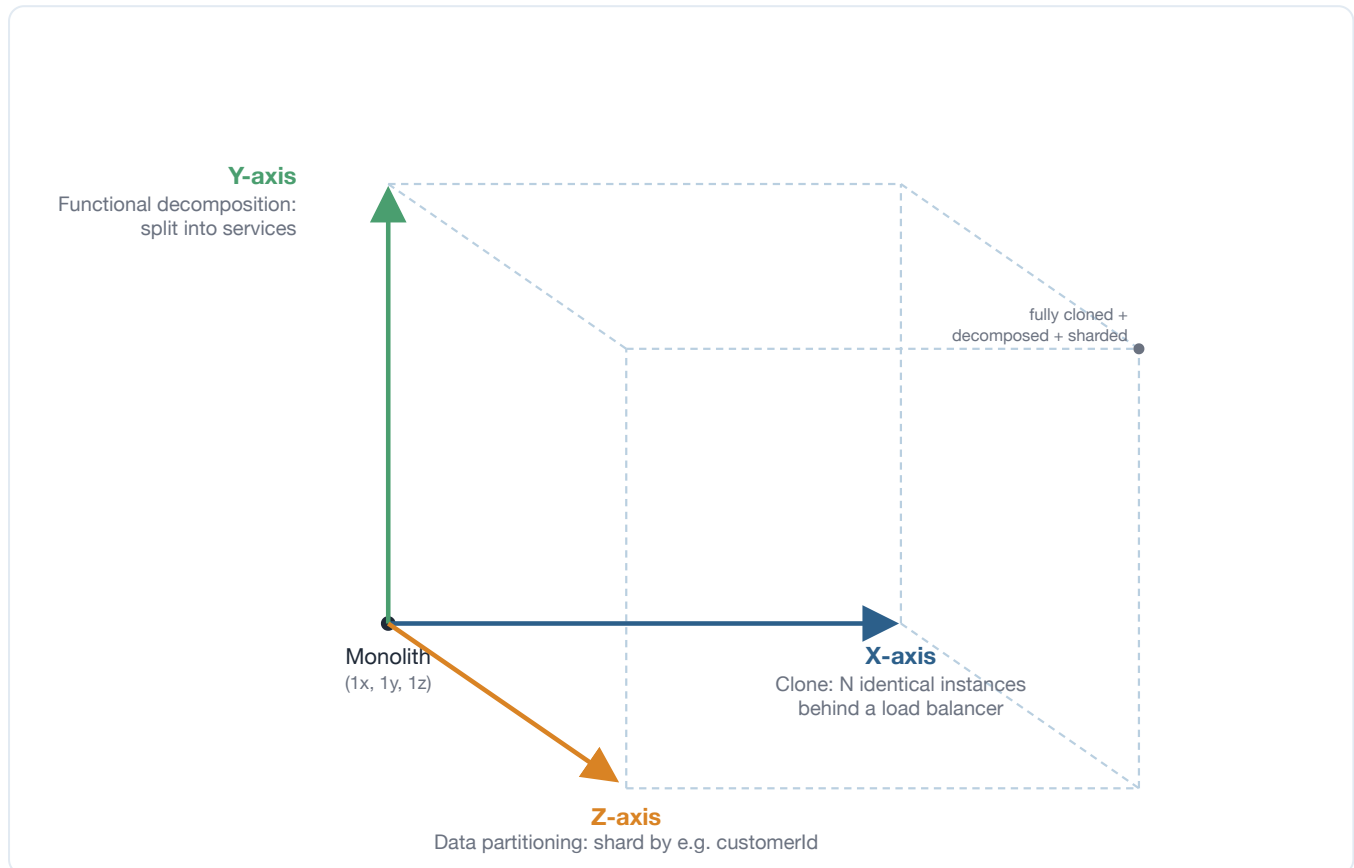


Figure 1.1 — The Scale Cube: three independent axes for scaling a system.

X-axis (clone) is the easy button: run many identical copies of the whole app behind a load balancer. It increases capacity and availability but does nothing for team scaling or codebase complexity — every copy is still the same giant monolith. **Z-axis (partition by data)** splits requests by some key (customerId, region) so each instance only handles a subset of data — great for very large datasets, but again each shard still runs the whole monolith. **Y-axis (functional decomposition)** is the one that matters for this book: split the application by function into separate services, each owning its own data. Microservice architecture is the application of the Y-axis; each resulting service can then separately apply the X and Z axes.

Staff-eng lens: when someone proposes "let's do microservices," ask which axis they actually need. If the real problem is just "we need more throughput," X-axis cloning is far cheaper than a rewrite. Microservices earn their complexity tax when the bottleneck is *organizational* (too many people in one codebase) or when parts of the system have genuinely different scaling/reliability/technology needs — not simply because traffic grew.

Defining microservice architecture

KEY IDEA

A **microservice architecture** structures an application as a set of services that are: independently deployable, loosely coupled, organized around business capabilities, and each owning its own data. Services communicate only through APIs (network calls) — never by reaching into each other's database.

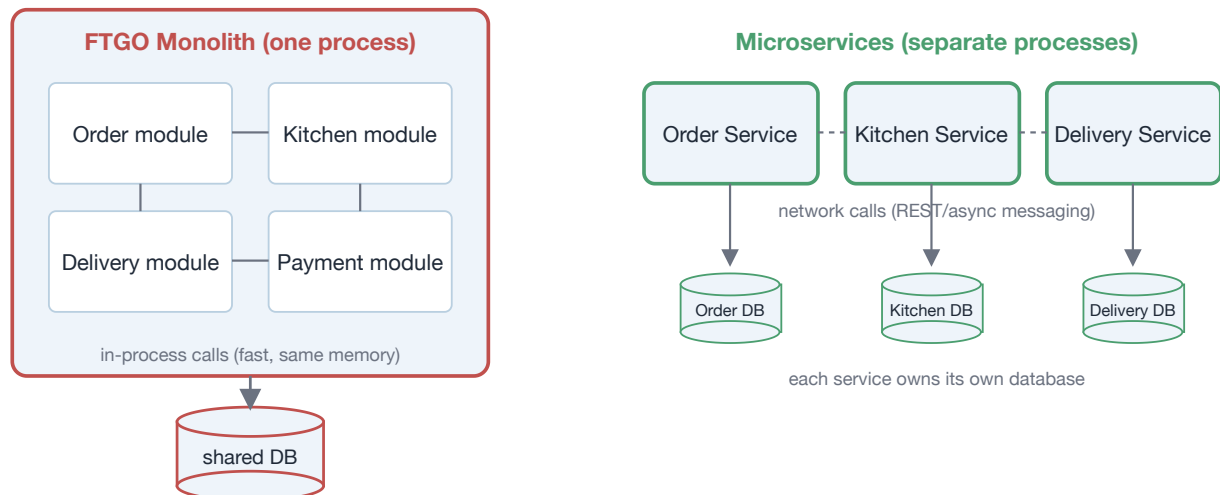


Figure 1.2 — Monolith (one process, one shared database) vs. microservices (separate processes, separate databases, communicating over the network).

Two design choices make this work: services must be **loosely coupled** (a change inside one service never forces a code change in another — it only exposes an API contract), and each service must have its own private, encapsulated data store so no other service can bypass its logic by touching its tables directly.

BENEFITS

Independent deployability (ship Order Service ten times a day without touching Delivery Service), small enough services that a small team can fully own one, freedom to pick the right tech per service (tech diversity), fault isolation (a Kitchen Service crash doesn't take down Payments), and the ability to scale only the services that actually need it.

TRADEOFFS / WATCH OUT

You now have a distributed system: network calls can fail or time out where a local function call couldn't. Finding the "right" service boundaries is genuinely hard and easy to get wrong. Data that used to live in one database with ACID transactions is now spread across many databases, so cross-service consistency needs new patterns (sagas, later chapters). Testing a flow that spans several services is harder than testing one process. And this only works with strong automation and DevOps practice — more moving parts means more to coordinate.

Analogy: a monolith is one big kitchen where anyone can grab any pot. Microservices are food trucks parked in a lot — each has its own inventory (database) and its own staff, and they can only "talk" to each other by shouting orders across the lot (network calls), not by walking into each other's truck.

A pattern language for microservices

The book is organized as a **pattern language**: a connected web of named, reusable solutions where solving one problem with a pattern typically introduces a new problem, solved by the next pattern. E.g. decomposing into services (this chapter/ch.2) creates a data-consistency problem, solved by Sagas; which creates a query problem, solved by API Composition/CQRS; and so on. Treat the book less like a list of tips and more like a decision tree you walk depending on which problem you actually have.

Process and organization

Microservices are as much an organizational bet as a technical one. **Conway's Law** says a system's structure ends up mirroring the communication structure of the organization that builds it — so if you draw service boundaries that don't match your team boundaries, you'll fight the org chart forever. This is the thinking behind **Team Topologies**-style organization: small, autonomous, cross-functional teams (each capable of building, testing, and operating their own services) aligned one-to-one (or few-to-one) with services. Microservices also assume a **DevOps** culture — the team that writes a service also runs it in production — and rely on **continuous delivery/deployment** pipelines, because with dozens of independently versioned services, manual release processes simply do not scale.

Staff-eng lens: in interviews and in real design reviews, the strongest answer to "should we do microservices?" is almost never purely technical. Ask: do we have (or can we build) team boundaries that map to proposed service boundaries? Do we have CI/CD, service-level monitoring, and on-call maturity? If the answer is no, adopting microservices first often means paying the distributed-systems tax without the organizational payoff.

Pattern: Monolithic architecture

Problem: you need to build and ship an application, ideally quickly and simply.

Solution: package the entire application — all business logic, all modules — as a single deployable unit (one process/container) backed by one database.

When to use: new products, small teams, early-stage startups, or systems with modest scale and a single, cohesive domain. Avoid once the team or codebase has outgrown "everyone understands everything."

Pattern: **Microservice architecture**

Problem: a monolith has become too large to develop, test, and deploy quickly and safely; different parts of the system need different scaling, reliability, or technology choices; and multiple teams keep blocking each other in one codebase.

Solution: structure the application as a set of loosely coupled, independently deployable services, each built around a business capability and owning its own data, communicating only via network APIs.

When to use: large, complex, long-lived applications built by multiple teams that need independent deployability and scalability — and only once you have the DevOps/CI-CD maturity to run many services in production.

Chapter 1 said "split the monolith into services organized around business capabilities" — this chapter is the how. It gives you a repeatable 3-step process for turning requirements into a set of service boundaries, two concrete patterns for drawing those boundaries (business capability, and DDD subdomains), two guiding principles for judging whether a boundary is good, and an honest look at the obstacles you'll hit once the single Order class in your monolith has to become several smaller things. This is the chapter to reread before any real service-boundary design review.

What is software architecture, anyway?

Architecture is the set of structures needed to reason about a software system: its elements, how they relate, and their properties. Richardson uses the **4+1 view model** (Philippe Kruchten) to organize the different structures:

- **Logical view** — the design that supports the application's functionality: for object-oriented systems, the classes and their relationships (this is where "services" live as a concept).
- **Implementation view** — how the code is organized into modules/packages/build artifacts.
- **Process view** — the runtime: processes, threads, and how they communicate — for microservices this is literally "which service calls which."
- **Deployment view** — how the processes map onto physical or virtual infrastructure (machines, containers, clusters).
- **"+1" Scenarios** — a handful of key use cases used to validate that the other four views actually work together.

Microservice architecture is mostly a statement about the logical view (decomposition into services) and the deployment view (independently deployable units) — but a good architecture has to make sense across all four.

Why architecture matters: the "-ilities"

Architecture matters because it determines whether a system can satisfy its non-functional requirements — the "-ilities" — not just its features. The ones the book highlights most: **scalability** (can it handle more load?), **reliability** (does it keep working when parts fail?), **maintainability** (can engineers keep changing it safely?), **testability** (can you verify it works, cheaply and often?), and **deployability** (can you ship changes quickly and safely?). Microservices are, fundamentally, a bet that decomposition improves maintainability and deployability enough to be worth the cost to simplicity.

Defining the microservice architecture: a 3-step process

Richardson gives a concrete, repeatable process for turning a set of requirements into a microservice architecture:

Step 1 — Identify system operations. Build a high-level domain model from the requirements (key nouns: Order, Restaurant, Courier...), then identify the **system operations** — the commands and queries the system must support — by walking through the use cases/user stories.


```
// A system operation identified from a use case:  
createOrder(consumerId, paymentMethodId, deliveryAddress,  
            restaurantId, lineItems): OrderInfo
```

Step 2 — Define the services. Group the domain model and the system operations into services, using a decomposition pattern (business capability and/or subdomain — see below) so each system operation is handled by one primary service.

Step 3 — Define each service's API and collaborations. Some system operations can be handled entirely inside one service. Others need several services to collaborate (e.g. **createOrder** touches Order, Kitchen, Accounting, and Consumer). For those, decide which service orchestrates the collaboration and design the APIs (sync or async) each service exposes — this is where later chapters on sagas and inter-service communication plug in.

Decompose by business capability

A **business capability** is something a business does to produce value, largely independent of how it's organized or implemented — e.g. "order management," "restaurant management," "delivery management," "accounting." You identify capabilities by analyzing the business itself (often they line up with departments), and each capability becomes a strong candidate for one service.

Analogy: imagine redrawing your company's org chart from scratch based only on "what does this team fundamentally do for the business" — ignoring current reporting lines. Business-capability decomposition does the same thing for services.

Decompose by subdomain

The business-capability approach can feel fuzzy for large, tangled domains, so the book brings in **Domain-Driven Design** (DDD). DDD splits the overall problem domain into **subdomains**, each with its own model, and each subdomain is implemented as one or more **bounded contexts** — a boundary inside which a specific model and its terms have one unambiguous meaning. In FTGO, "Order" means something different in Order-taking, Kitchen, and Delivery subdomains; DDD says: don't force one shared "Order" class to serve all three — give each bounded context/service its own local model.

EXAMPLE

FTGO's subdomains map roughly to: Order taking & management → Order Service; Kitchen operations → Kitchen Service; Delivery/logistics → Delivery Service; Restaurant onboarding & menus → Restaurant Service; Consumer profile & management → Consumer Service; Billing → Accounting Service. Business-capability analysis and subdomain analysis usually converge on a very similar set of services — they're two lenses on the same underlying structure.

Decomposition guidelines

Two classic object-oriented design principles, applied at the service level, help judge whether boundaries are good:

Single Responsibility Principle (SRP): a class (or here, a service) should have only one reason to change. Applied to services, this pushes you toward small, cohesive services rather than a service that owns "orders

and also payments and also notifications."

Common Closure Principle (CCP): classes that tend to change for the same reason, at the same time, should live together. Applied to services, this means: if two pieces of logic almost always change together, put them in the same service — otherwise one business change forces a synchronized deploy of two services, which defeats independent deployability.

KEY IDEA

SRP pulls toward smaller services (do one thing). CCP pulls toward grouping things that change together (avoid multi-service deploys for one change). Good decomposition is the balance point between the two — not "as many tiny services as possible."

Obstacles to decomposition

Splitting a monolith isn't free. Three obstacles show up immediately:

TRADEOFFS / WATCH OUT

Network latency. A method call that used to be a nanosecond in-process call is now a network round trip. Too many fine-grained inter-service calls to satisfy one request can make the system slower than the monolith it replaced. Fix: design coarser-grained APIs, batch calls, or restructure so one service can answer without asking three others.

Reduced availability from synchronous calls. If handling a request requires service A to synchronously call B, then C, then D, the overall availability is roughly the *product* of each service's individual availability — which is always lower than any single service's availability. Chaining synchronous calls multiplies your risk of failure. Fix: prefer asynchronous messaging, or trim genuinely unnecessary synchronous dependencies.

Maintaining data consistency across services. An operation that used to be one local ACID transaction across a few tables in one database now has to update data owned by several independent services with independent databases. You can't just wrap it in one transaction anymore — you need patterns like Sagas (covered later in the book).

The "god class" problem: Order

In the FTGO monolith, the **Order** class is a classic **god class**: over time it accumulated fields and behavior needed by order-taking, the kitchen, delivery, and accounting — everyone touched it, so it grew to know about everything. When you decompose by subdomain, you cannot keep one shared Order class used by every service, because that would recreate tight coupling between services (exactly what microservices are supposed to remove). The DDD fix is to give each service its *own*, smaller, local model of the concept it actually needs — Order Service has a full Order; Kitchen Service only needs enough to build a Ticket; Delivery Service only needs a Delivery/pickup-dropoff view — kept consistent, when needed, via events rather than a shared class.

Staff-eng lens: "god classes" are usually the single biggest tell that a monolith's internal structure never had real module boundaries. When you're asked to review a decomposition proposal, look for any entity everyone imports (Order, User, Account) — that's exactly where naive service boundaries will leak, and exactly where you should push for per-service, purpose-built models instead of a shared "God" DTO passed everywhere.

FTGO decomposed

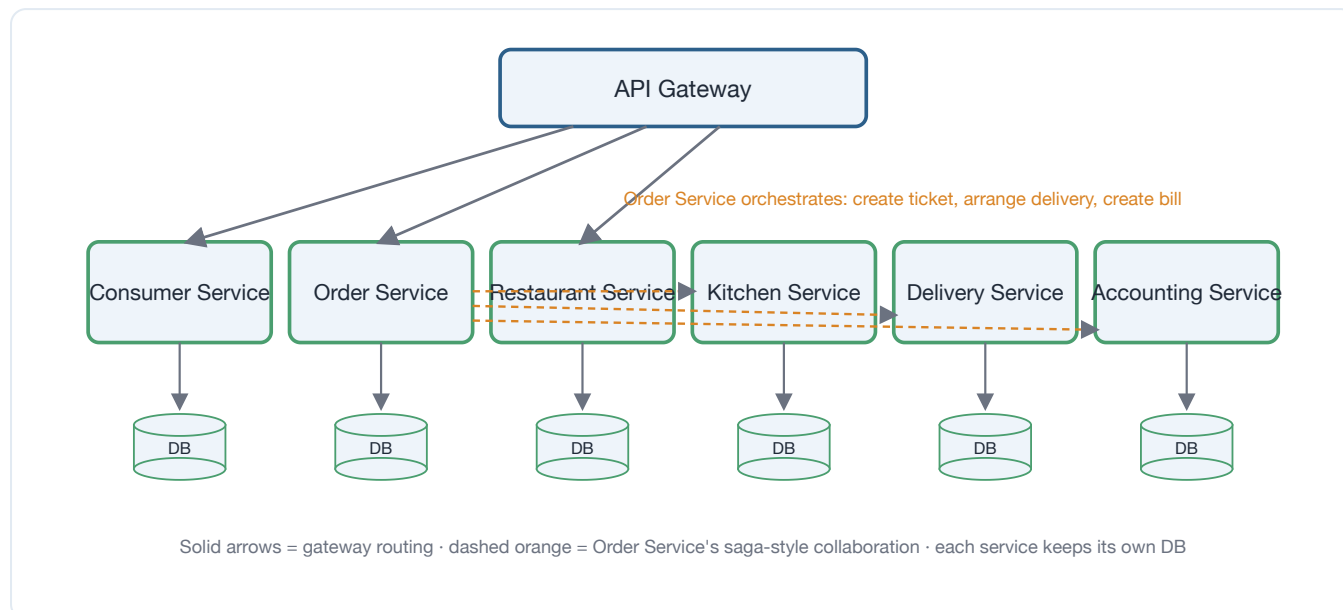


Figure 2.1 — FTGO decomposed: API Gateway routes to the front-line services; Order Service collaborates with Kitchen, Delivery, and Accounting to fulfil an order.

From business capabilities to services

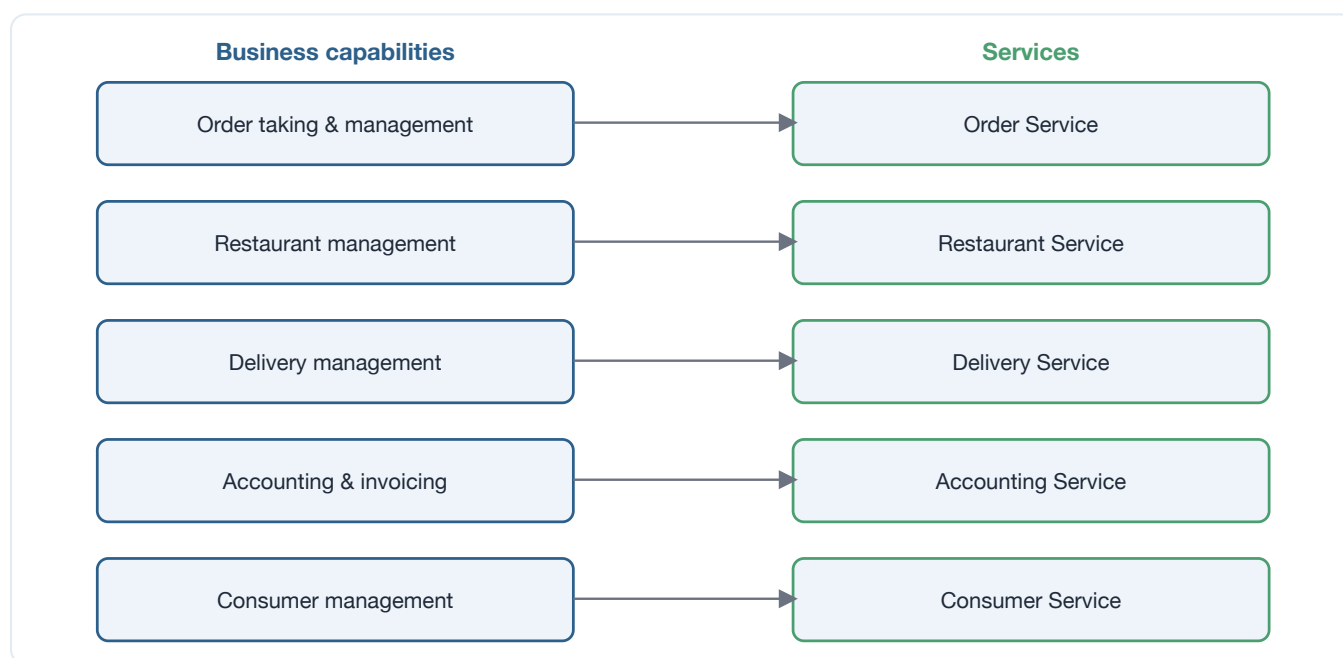


Figure 2.2 — Each identified business capability maps to one candidate service.

Pattern: **Decompose by business capability**

Problem: how do you divide an application into services in a way that's stable and aligned with how the business actually operates (not with today's org chart or technical layers)?

Solution: identify the business's capabilities — the things it does to generate value (order management, restaurant management, delivery management, accounting) — usually by analyzing the business itself, and turn each capability into a candidate service.

When to use: as a first pass on almost any decomposition effort; works especially well when the business's capabilities are already reasonably well understood and distinct.

Pattern: **Decompose by subdomain**

Problem: business-capability analysis feels too coarse or ambiguous for a large, tangled domain — you need a more rigorous way to draw boundaries, especially around shared concepts like "Order" that mean different things to different parts of the business.

Solution: apply Domain-Driven Design: split the overall domain into subdomains, each with its own model, and implement each subdomain as one or more bounded contexts (services), each with its own local version of shared concepts instead of one god class.

When to use: complex domains with rich business logic, or when business-capability analysis produces overlapping/ambiguous capabilities that DDD's subdomain/bounded-context lens can disambiguate.

03 Interprocess communication in a microservice architecture

In a monolith, modules talk to each other with plain function calls. In a microservice architecture, every one of those calls crosses a network. Networks are slow, flaky, and sometimes just gone. This chapter is about choosing how services talk (sync request/response vs async messages), how to define and evolve their APIs without breaking callers, and how to survive when a downstream service is slow or down. A staff engineer needs this chapter cold — IPC choice is one of the first architecture decisions that locks in your system's availability ceiling.

Interaction styles

There are two independent questions to answer for every service-to-service interaction: **how many participants** (one-to-one or one-to-many), and **does the caller wait** (synchronous or asynchronous).

KEY IDEA

One-to-one: one client, one service. **One-to-many:** one client, many interested services.

Synchronous: the client blocks and expects a timely reply. **Asynchronous:** the client doesn't block; it may not expect a reply at all, or the reply arrives later on its own channel.

Mixing and matching the two axes gives the interaction styles the book lists:

- **Request/response (one-to-one, sync)** — Order Service calls Kitchen Service and waits for the answer. Classic REST or RPC call.
- **Async request/response (one-to-one, async)** — Order Service sends a request and continues; the reply comes back later on a separate channel.
- **One-way notifications (one-to-one, async)** — Order Service sends a message and doesn't expect any reply at all, e.g. "send this receipt email."
- **Publish/subscribe (one-to-many, async)** — Order Service publishes **OrderCreated**, and Kitchen Service, Delivery Service, and Consumer Service each react independently.
- **Publish/async responses (one-to-many, async)** — Order Service publishes a request and gathers replies from multiple services, e.g. asking Kitchen Service and Delivery Service each to confirm availability.

Analogy: synchronous request/response is a phone call — you stay on the line until you get an answer. Publish/subscribe is a company-wide Slack announcement — you post once, and everyone who cares reacts on their own time.

Staff-eng lens: the interaction style is a design decision, not an implementation detail. Picking sync request/response everywhere is the single most common reason microservice migrations end up less available than the monolith they replaced — you've just added network hops to every call chain.

Defining APIs and keeping them alive

Every service has an API: the operations it accepts, the events it publishes, the messages it consumes. In **API-first design**, you write the API specification (an OpenAPI/Swagger doc for REST, a `.proto` file for gRPC, an AsyncAPI doc for events) before writing the implementation, and you review it with the teams that will consume it. This turns an integration problem into a contract negotiation that happens on a whiteboard, not in production.

EXAMPLE

Kitchen Service's API might expose `createTicket()`, `acceptTicket()`, and publish a `TicketAcceptedEvent`. Writing the OpenAPI spec first lets Order Service's team start building against a mock before Kitchen Service's real code exists.

APIs inevitably change. The hard problem is **evolving** them without breaking clients you don't control and can't force to redeploy in lockstep.

- **Backward-compatible (minor) changes** — adding an optional field, adding a new operation. Old clients keep working unchanged.
- **Breaking (major) changes** — removing/renaming a field, changing a field's meaning or type. Old clients break.

Use **semantic versioning** (MAJOR.MINOR.PATCH) to signal which kind of change shipped: bump MAJOR only for breaking changes, MINOR for backward-compatible additions, PATCH for bug fixes. Clients then know exactly when they must act.

On the consuming side, apply the **Robustness Principle** ("be liberal in what you accept") by writing a **tolerant reader**: ignore fields you don't recognize, don't fail on missing optional fields, extract only the data you actually need instead of binding to the whole payload. A tolerant reader survives many changes the producer thought were "breaking."

Staff-eng lens: when you can't force every consumer to upgrade simultaneously (you almost never can, across org boundaries), run old and new major versions side by side for a deprecation window, and make "tolerant reader" a code-review checklist item for anyone consuming another team's event or API.

Message formats: text vs binary

Once you know the interaction style, you need a wire format for the actual bytes.

- **Text formats** — JSON, XML. Human-readable, easy to debug with curl, naturally flexible (extra fields are just ignored), but larger on the wire and slower to parse.
- **Binary formats** — Protocol Buffers, Avro. Compact and fast, and — importantly — they come with a schema (a `.proto` or Avro schema file) that gives you built-in, checked backward/forward compatibility rules instead of "hope everyone reads the docs."

Staff-eng lens: JSON is the right default for public/partner-facing APIs where debuggability and broad tooling support matter more than a few milliseconds. Binary formats with schemas earn their complexity inside a high-throughput internal call graph, or anywhere you want the schema itself to enforce compatibility at build time.

Remote Procedure Invocation (RPI)

Pattern: Remote procedure invocation

Problem: a client needs to invoke an operation on a remote service and get a result, using an interaction style that feels like a normal method call.

Solution: the client uses a request/response protocol — typically REST over HTTP or gRPC over HTTP/2 — to call the service directly (or via a load balancer), and the service returns a response.

When to use: when the caller genuinely needs an immediate answer to proceed (a query, or a command with a required synchronous result), and the extra availability coupling from a direct network call is acceptable.

REST and the Richardson Maturity Model

REST models the API as a set of resources (URLs, nouns) manipulated with HTTP verbs (GET/POST/PUT/DELETE). The **Richardson Maturity Model** (no relation to the book's author, just a naming coincidence) grades how "RESTful" an API really is:

- **Level 0** — one URL, everything goes through POST (basically RPC-over-HTTP, e.g. a single `/api` endpoint).
- **Level 1** — multiple URLs, one per resource (e.g. `/orders`, `/tickets`), but still mostly POST.
- **Level 2** — proper use of HTTP verbs and status codes: GET to read, POST to create, PUT to update, 404/409/etc. for errors. Most "REST APIs" in the wild stop here.
- **Level 3** — adds **HATEOAS** (Hypertext As The Engine Of Application State): responses include links describing what you can do next, so clients discover valid actions instead of hardcoding URLs.

BENEFITS OF REST

Simple and familiar; no broker to run; request and response can be observed and cached with standard HTTP tooling (proxies, browsers, curl).

TRADEOFFS / WATCH OUT

Only supports request/response, so it can't express one-to-many or fire-and-forget cleanly. It couples the availability of the caller to the availability of the callee. And it can lead to under- or over-fetching, pushing teams toward API composition layers.

gRPC

gRPC is an RPC framework built on HTTP/2 and Protocol Buffers. You define the service and its messages in a `.proto` file, and gRPC generates strongly-typed client and server stubs in multiple languages.

```
// kitchen.proto
service KitchenService {
  rpc CreateTicket (CreateTicketRequest) returns (CreateTicketReply);
}
message CreateTicketRequest {
  string restaurantId = 1;
  string orderId = 2;
}
```

gRPC is efficient (binary, HTTP/2 multiplexing) and supports streaming (client, server, or bidirectional), which plain REST does not do well. The cost is that it's less browser/tooling-friendly than JSON-over-HTTP and requires the shared .proto contract to be distributed to all clients.

Handling partial failure

Any synchronous call can fail because the callee is slow, overloaded, or dead. Without protection, one struggling service creates a cascading pile-up of blocked threads in every service that calls it — this is exactly how a small outage becomes a full outage.

Pattern: Circuit breaker

Problem: a remote call to a failing service keeps being retried/blocked, exhausting the caller's threads/connections and taking the caller down too.

Solution: wrap the remote call in a proxy that tracks recent failures. Once failures cross a threshold, the proxy "trips" and starts failing fast without even attempting the call, giving the downstream service room to recover, and giving the caller a chance to run a fallback.

When to use: for every network call to another service, essentially always — this is baseline hygiene in a microservice architecture, not an optional extra.

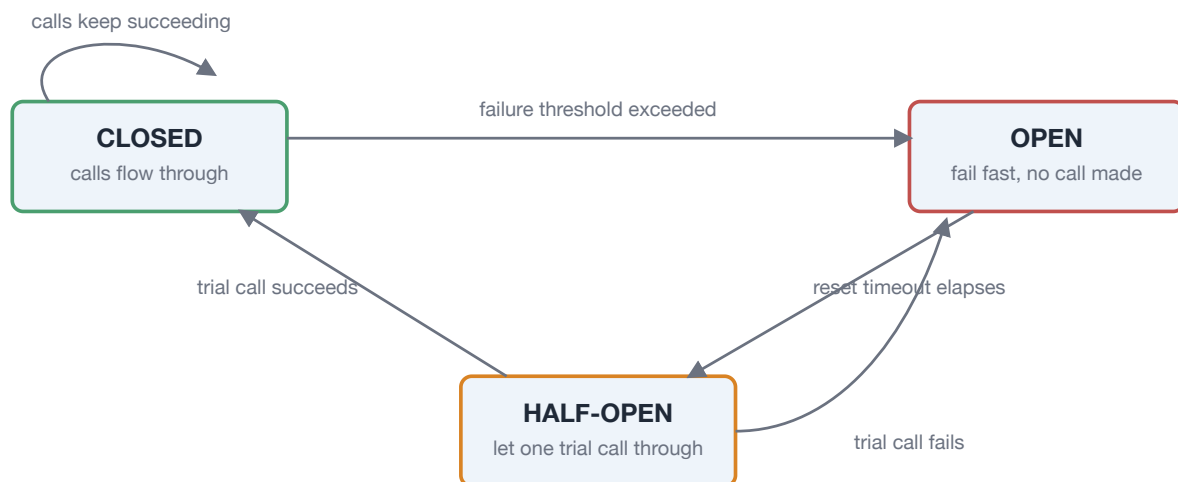


Figure 3.1 — Circuit breaker state machine: Closed → Open → Half-Open.

Netflix's **Hystrix** library popularized this: it wraps each remote call, tracks a rolling window of successes/failures/timeouts, trips the breaker when the error rate crosses a threshold, and every call executes inside its own small thread pool (or semaphore) so one overloaded dependency can't starve threads needed by calls to healthy dependencies.

Circuit breakers are one tool among several for handling partial failure:

- **Timeouts** — never wait forever; bound every network call so a hung dependency can't hang you.
- **Retries with exponential backoff** — retry transient failures, but wait longer between each attempt (and add jitter) so a struggling service isn't hit with a retry storm right when it's trying to recover.

- **Bulkheads** — like the watertight compartments in a ship's hull, give each downstream dependency its own thread pool/connection pool so a flood in one compartment doesn't sink the whole ship.

When a call ultimately fails (breaker open, timeout, retries exhausted), the caller needs a recovery strategy:

- **Return an error** — honest, but pushes the problem to whoever called you.
- **Return a fallback value** — a sensible default, e.g. "assume delivery in 30 minutes" if Delivery Service is unreachable.
- **Return cached data** — slightly stale is often better than nothing, e.g. cached restaurant menu.

Staff-eng lens: in a design review, "what happens when this dependency is down" is the question that separates a working architecture from a demo. Push every synchronous dependency to declare its timeout, its fallback, and whether it even needs to be synchronous at all.

Service discovery

In the cloud, service instances get dynamically assigned IPs, scale up and down, and get replaced constantly. A hardcoded host:port list breaks within minutes. **Service discovery** solves "given a logical service name, find a live instance to call right now."

There are two families of solutions:

Pattern: Self-registration

Problem: a service instance's location must be knowable by clients, and instances come and go.

Solution: each service instance registers itself with a service registry on startup, sends periodic heartbeats to prove it's alive, and deregisters (or is timed out) on shutdown.

When to use: when you're building the discovery mechanism into the application layer yourself, typically paired with client-side discovery.

Pattern: Client-side discovery

Problem: the calling service needs to pick a specific healthy instance to call, ideally with load balancing.

Solution: the client queries the service registry (e.g. Netflix Eureka) directly, gets back a list of live instances, and picks one itself (often round-robin) before making the call.

When to use: when clients are willing to embed a discovery/load-balancing library, in exchange for one less network hop per call.

Pattern: 3rd party registration

Problem: you don't want every service to implement self-registration logic.

Solution: a separate registrar process (often built into the deployment platform) watches the environment and registers/deregisters instances in the registry on their behalf.

When to use: when the platform already tracks instance lifecycle (e.g. a Kubernetes controller, or a registrar sidecar), so application code stays discovery-agnostic.

Pattern: Server-side discovery

Problem: same as above, but you'd rather clients not know anything about discovery at all.

Solution: the client calls a stable endpoint — a router or load balancer (AWS ELB, a Kubernetes Service + kube-proxy/DNS) — which looks up healthy instances itself and forwards the request.

When to use: the default on modern platforms. Kubernetes and most cloud load balancers give you this for free, which is why hand-rolled client-side discovery has become less common than it was when the book was written.

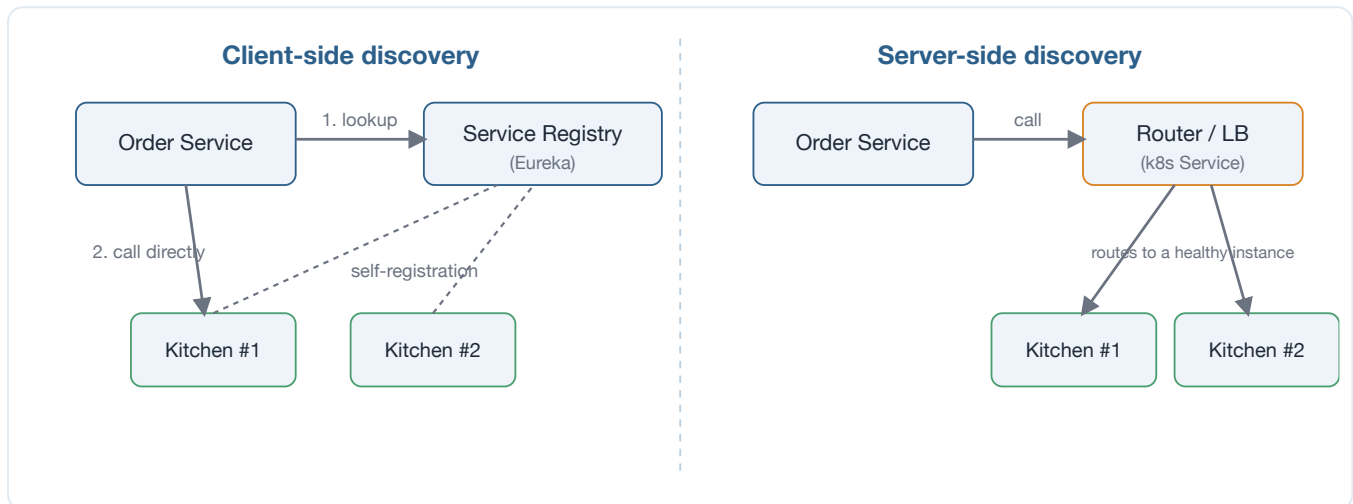


Figure 3.2 — Client-side discovery (client queries the registry itself) vs server-side discovery (a router does it).

Common registries: **Netflix Eureka** (client-side style, AP-oriented), **HashiCorp Consul** and **etcd** (also used as Kubernetes' own backing store, CP-oriented). Kubernetes and AWS ELB give you server-side discovery baked into the platform, which is why most teams building on Kubernetes today never touch a registry API directly.

Staff-eng lens: know which pattern your platform already gives you for free before proposing to add Eureka/Consul. On Kubernetes, server-side discovery via Services/DNS is usually the pragmatic default; application-level discovery earns its keep mainly when you need smarter client-side load-balancing logic (e.g. zone-aware routing) than the platform's router provides.

Asynchronous messaging

Pattern: Messaging

Problem: services need to communicate reliably without the caller blocking on the callee being up and responsive right now.

Solution: services exchange messages asynchronously via a message broker (Apache Kafka, RabbitMQ, ActiveMQ), using a **channel** — point-to-point or publish-subscribe — instead of calling each other directly.

When to use: for commands where the sender doesn't need an immediate answer, and always for events ("something happened") that other services might care about.

Channels come in two shapes:

- **Point-to-point** — exactly one consumer processes each message (e.g. an order command queue read by one Kitchen Service instance).
- **Publish-subscribe** — every subscriber gets its own copy of each message (e.g. an OrderCreated topic read independently by Kitchen Service, Delivery Service, and Consumer Service).

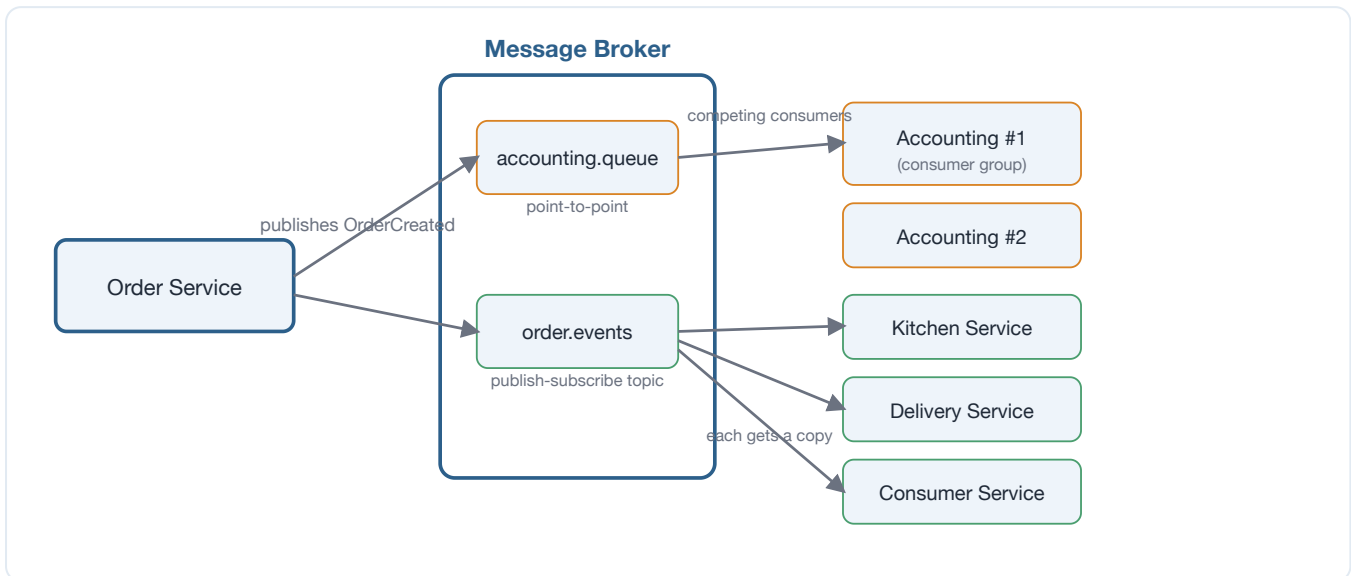


Figure 3.3 — One broker, two channel shapes: a point-to-point queue with competing consumers, and a publish-subscribe topic fanning out to independent subscribers.

BENEFITS

Loose coupling (publisher doesn't know or care who's listening); the broker buffers messages so a slow/down consumer doesn't block the producer; this generally raises overall availability since the sender only needs the broker to be up, not every downstream service.

TRADEOFFS / WATCH OUT

You now operate and scale a broker — one more piece of critical infrastructure. Request/response over messaging is awkward (needs correlation ids and a reply channel). And you inherit new distributed-systems problems: ordering, duplicate delivery, and exactly-once-feeling processing.

Ordering, competing consumers, and duplicates

Most brokers only guarantee order within a single partition/shard of a channel, not across the whole topic. Kafka handles this by letting you pick a **partition key** (e.g. orderId) — all messages for the same order land in the same partition and are delivered in order, while different orders can be processed in parallel across partitions.

Within a point-to-point queue, multiple consumer instances (a **competing consumers** group) can process messages in parallel to scale throughput — but that means two different messages for the same entity could, in principle, be handled by two instances at once, which is another reason to partition by entity id.

Brokers typically offer **at-least-once** delivery, meaning the same message can arrive twice (e.g. after a consumer crash right before it acknowledges). The fix is to make message handlers **idempotent**: processing the same message N times has the same effect as processing it once.

```
// idempotent handler: skip if we've already processed this message id
if (processedMessageRepository.exists(message.getId())) {
    return; // duplicate – already handled
}
applyOrderCreated(message);
processedMessageRepository.save(message.getId());
```

Staff-eng lens: "idempotent by message id" is a cheap, general-purpose defense. It's the async-messaging equivalent of a REST API's idempotency key, and it's usually far simpler than trying to get true exactly-once delivery out of the broker.

Transactional messaging

Here's a trap: a request handler needs to both update its own database *and* publish an event about it (e.g. Order Service saves the new Order row and publishes `OrderCreated`). If it does the DB write and the broker publish as two separate steps, one can succeed while the other fails — this is the **dual-write problem**. A crash between the two steps leaves your data and your events permanently inconsistent, and no ordinary database transaction spans both a database and a broker.

Pattern: Transactional outbox

Problem: a service must atomically update its database and publish a message, but the database and the broker can't share one transaction.

Solution: in the same local database transaction as the business update, insert the message into an OUTBOX table in that same database. A separate step then reads the outbox and publishes to the broker. Since both writes are one ACID transaction, they either both happen or neither does.

When to use: essentially always, whenever a service needs to reliably publish an event as a side effect of a state change — this is the standard building block, and other patterns below are just ways to drain the outbox.

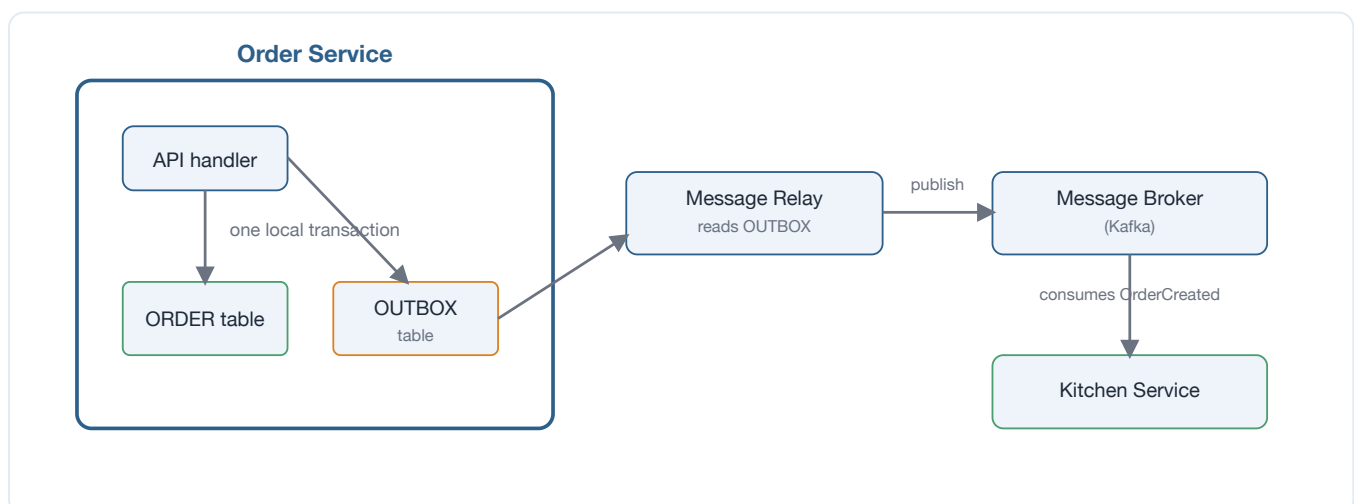


Figure 3.4 — Transactional outbox: the DB update and the outbox row commit together; a relay drains the outbox to the broker.

Two common ways to implement the relay that drains the outbox:

Pattern: Polling publisher

Problem: something has to move rows from the OUTBOX table to the broker.

Solution: a relay process periodically polls the OUTBOX table (`SELECT ... WHERE published = false`), publishes each row to the broker, and marks it published (or deletes it).

When to use: simplest option, fine for moderate throughput; the tradeoff is polling latency and load on the database.

Pattern: Transaction log tailing

Problem: polling adds latency and repeated query load just to notice new outbox rows.

Solution: tail the database's own transaction/commit log (Change Data Capture) and turn each committed insert into the OUTBOX table into a published message — no polling query needed. **Debezium** is the standard open-source tool for this against MySQL/Postgres/etc.

When to use: higher-throughput or lower-latency needs, and where you're willing to run/operate a CDC pipeline as extra infrastructure.

Analogy: the outbox table is a restaurant's order ticket rail — the kitchen (your DB transaction) prints the ticket and hangs it on the rail in the same motion as accepting the order. The relay (or a CDC tailer watching the rail) is the runner who picks up printed tickets and carries them out — the order and the "someone needs to act on this" signal can never get out of sync.

Async messaging as an availability lever

Every synchronous call you add to a request path multiplies your unavailability: if Order Service calls three downstream services synchronously and each is 99.9% available, the composite availability of that call is roughly $99.9\%^3 \approx 99.7\%$ — worse than any one of them alone. Synchronous chains turn "everyone is usually up" into "the whole chain is down whenever anyone is down."

Two techniques from the book for reducing this synchronous coupling:

- **Replicate the data you need instead of calling for it live** — e.g. Order Service keeps a local, denormalized, eventually-consistent copy of restaurant menu data (kept up to date via events from Restaurant Service) instead of calling Restaurant Service synchronously on every order.
- **Return before processing fully completes** — accept the request, durably record intent (often via the transactional outbox), and finish the actual work asynchronously, publishing an event or notifying the client later when it's done.

Staff-eng lens: "how many synchronous hops does this request path have" is a metric worth tracking, the same way you'd track P99 latency. Each one is a shared-fate dependency. Async messaging with the transactional outbox is how you convert "must be up" dependencies into "must eventually process" dependencies — a strictly weaker, and far cheaper, requirement.

KEY IDEA

Chapter 3 in one line: pick the interaction style deliberately, defend every synchronous call with timeouts/circuit breakers/bulkheads, let the platform handle discovery where it can, and reach for async messaging plus the transactional outbox whenever you don't truly need an immediate answer — because every synchronous hop you remove is availability you get back for free.

In a monolith, one database transaction can touch many tables and either all of it commits or none of it does. In microservices, every service guards its own database, so a single business operation (like placing a food order) has to update several databases owned by different services. This chapter is about **Sagas** — the pattern that keeps that data consistent without a giant distributed transaction. A staff engineer needs this chapter cold: it's the pattern interviewers probe hardest, and getting it wrong in production means double-charged customers or orders stuck forever in "pending".

The problem: no more one big ACID transaction

In a monolith, "place an order" might be one **ACID** transaction: check the consumer, create the order row, create a kitchen ticket row, charge the card — all in one commit. If anything fails, the database rolls everything back automatically.

In FTGO's microservices architecture, **Order Service**, **Consumer Service**, **Kitchen Service**, and **Accounting Service** each own a private database. There is no single database to open one transaction against. You need a way to keep data consistent *across* services.

WHY NOT JUST USE A DISTRIBUTED TRANSACTION (2PC)?

Two-phase commit (2PC) lets a coordinator ask every participant "can you commit?" (phase 1), and only if everyone says yes does it tell them all to actually commit (phase 2). It sounds like the obvious fix, but it's avoided in practice:

Availability: 2PC is a blocking protocol — every participant holds locks and waits for the coordinator. If one service is slow or down, all the others sit there holding locks. This directly fights the availability that microservices are supposed to buy you.

Technology support: many modern databases and, importantly, most message brokers (Kafka, RabbitMQ) and NoSQL stores don't support the two-phase-commit standard (XA) at all. If your architecture leans on messaging or NoSQL — and most microservice architectures do — 2PC is often not even an option.

The solution: Saga — a chain of local transactions

A **saga** replaces one big distributed transaction with a sequence of small **local transactions**. Each local transaction updates the database of just one service, then publishes a message or event that triggers the next local transaction in the next service. If a step fails, the saga runs **compensating transactions** to semantically undo the steps that already completed.

KEY IDEA

A saga is: local transaction → event/message → local transaction → event/message → ... If step N fails, don't roll back the database (you can't, other services already committed) — instead run compensating transactions for steps 1..N-1 to undo their business effect.

Analogy: booking a trip means booking a flight, a hotel, and a rental car — three separate reservations at three separate companies. There's no single "transaction" spanning all three. You book the flight, then the hotel, then the car. If the car booking fails, you don't get a magic database rollback — you call the airline and the hotel and *cancel* those bookings. Cancelling is the compensating transaction; it undoes the business effect, not the literal database write.

Example: the FTGO Create Order Saga

Placing an order in FTGO is one saga with six local transactions, spread across four services:

1. **Order Service** — `createOrder()`, saves the order with state `PENDING`.
2. **Consumer Service** — `verifyConsumerDetails()`, checks the consumer is allowed to order.
3. **Kitchen Service** — `createTicket()`, creates a ticket with state `CREATE_PENDING`.
4. **Accounting Service** — `authorizeCard()`, authorizes the consumer's credit card.
5. **Kitchen Service** — `approveTicket()`, moves the ticket to `AWAITING_ACCEPTANCE`.
6. **Order Service** — `approveOrder()`, marks the order `APPROVED`.

EXAMPLE — WHEN THE CARD IS DECLINED

If step 4 (authorize card) fails, the saga can't just stop — the ticket already exists in the Kitchen Service database. So it runs compensating transactions in reverse: Kitchen Service `rejectTicket()` (undoes the created ticket) and Order Service `rejectOrder()` (marks the order `REJECTED` so the consumer sees a clear result instead of a silently vanished order).

Compensatable, pivot, and retrievable transactions

Not every step in a saga is the same kind of step. Richardson splits them into three types, and knowing which is which tells you where the "point of no return" is:

- **Compensatable transaction** — can be undone by a compensating transaction. Example: `createTicket` is undone by `rejectTicket`.
- **Pivot transaction** — the go/no-go point of the saga. If it succeeds, the saga is guaranteed to run through to completion (no more failures are allowed to stop it). Example: `authorizeCard` — once the card is authorized, FTGO commits to fulfilling the order.
- **Retriable transaction** — comes after the pivot transaction, and is guaranteed to succeed (retried until it does, no business reason for it to fail). Example: `approveTicket`, `approveOrder`.

Staff-eng lens: when designing a saga, the first question to ask is "which step is the pivot?" Everything before it must be compensatable; everything after it must be made retrievable (idempotent, backed by retry-with-backoff). Get this classification wrong and you'll either try to compensate something that already can't be undone, or you'll leave a step that can silently fail after the point of no return.

Two ways to coordinate a saga

Someone has to decide "what happens next" after each local transaction. There are two styles.

Choreography — services react to each other's events, no coordinator

Each service publishes a domain event when it finishes its local transaction. Other services subscribe to events they care about, and react by running their own local transaction and publishing their own event. Nobody is "in charge" — the sequence emerges from services reacting to each other.

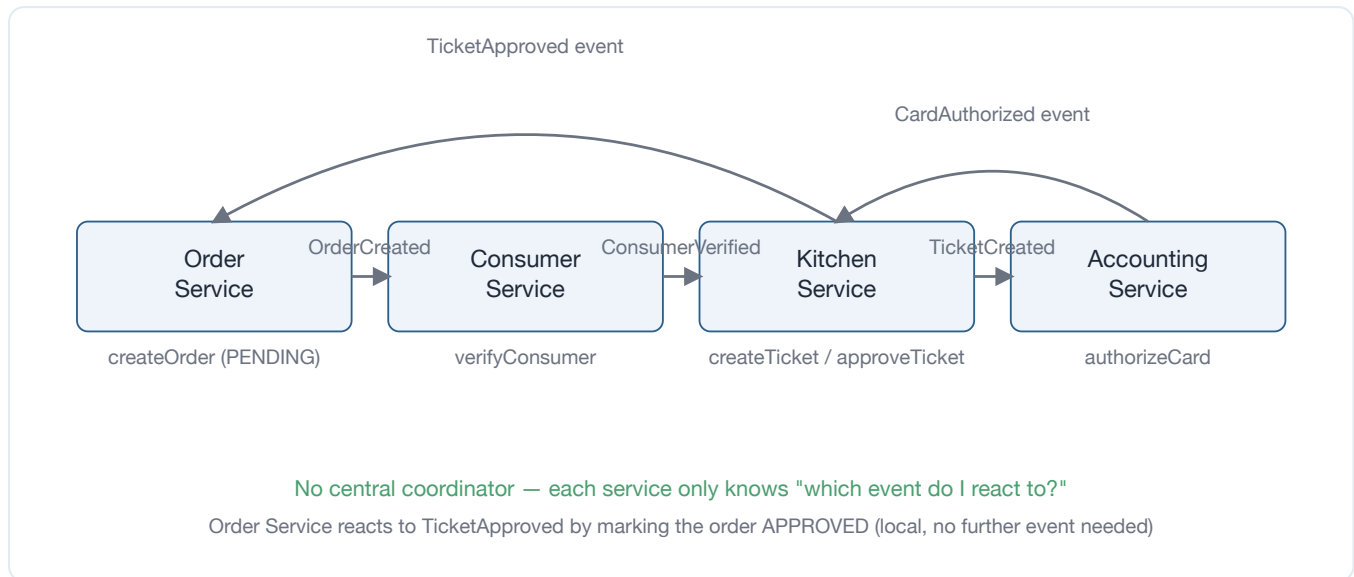


Figure 4.1 — Choreography-based Create Order Saga: services chained purely by events.

BENEFITS OF CHOREOGRAPHY

No single point of coordination logic to build or own. Services are loosely coupled — Order Service doesn't know Kitchen Service exists, it just knows about events. Simple to add a new participant: just have it subscribe to an existing event.

TRADEOFFS OF CHOREOGRAPHY

As the saga grows, the overall flow lives nowhere — it's smeared across every service's event handlers. It's hard to answer "what happens when the card authorization fails?" without reading code in four different repos. Risk of cyclic dependencies between events, and testing the whole flow means spinning up every participant.

Orchestration — a central saga orchestrator tells everyone what to do

An **orchestrator** is an object (often implemented as a state machine) that knows the whole saga. It sends **command messages** to each participant telling it which local transaction to run, and reacts to their reply messages by deciding the next command — including which compensating commands to send on failure.

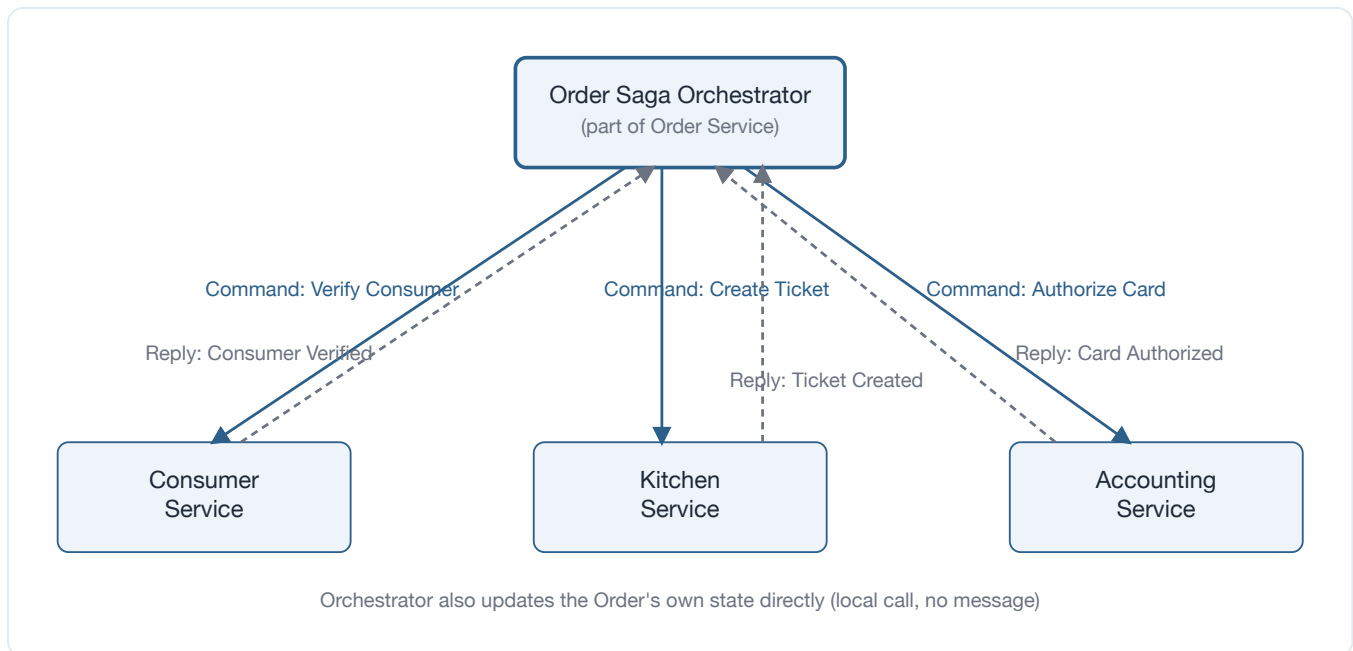


Figure 4.2 — Orchestration-based Create Order Saga: a central orchestrator drives each participant with commands and reads their replies.

BENEFITS OF ORCHESTRATION

The whole flow — including every failure and compensating path — is defined in one place: the orchestrator's state machine. That makes it far easier to understand, test, and reason about. Adding a step is a change in one file, not a hunt through every service.

TRADEOFFS OF ORCHESTRATION

The orchestrator now knows about (is coupled to) every participant and their commands — it centralizes logic, and centralized logic can become a design magnet ("just add one more field to the command") that grows into a mini-monolith if you're not careful. Someone has to own and operate that orchestrator.

Staff-eng lens: this is a coupling trade, not a right-or-wrong choice. Choreography trades a visible flow for loose coupling; orchestration trades some coupling (to the orchestrator) for a visible, testable flow. Rule of thumb used in practice: 2-3 participants and a simple happy path → choreography is fine. 4+ participants, conditional branches, or multiple failure/compensation paths → orchestration wins, because at that complexity "the flow lives in everyone's event handlers" becomes genuinely unreadable. In interviews, naming this trade-off explicitly (and picking based on step count / branching, not taste) is what separates senior from staff-level answers.

The Create Order Saga, step by step

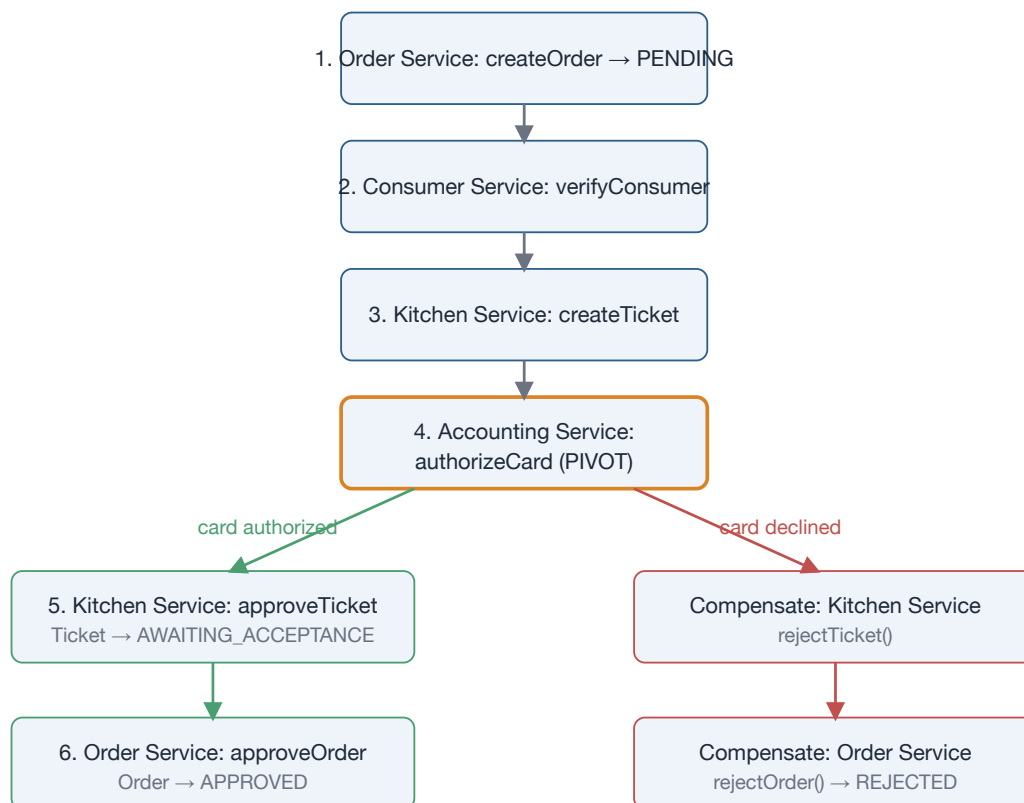


Figure 4.3 — Create Order Saga happy path (green, steps 1-6) and the compensating path if the credit card is declined at step 4 (red).

READING THE COMPENSATION PATH

Steps 1-3 (create order, verify consumer, create ticket) are **compensatable**. Step 4 (authorize card) is the **pivot** — success there means the saga commits to completing. If step 4 fails instead, the saga runs the compensating transactions for the steps that already ran: reject the ticket, then reject the order. Note there's no need to "compensate" step 2 (verifyConsumer) — it never wrote anything, so there's nothing to undo.

A tiny state machine view

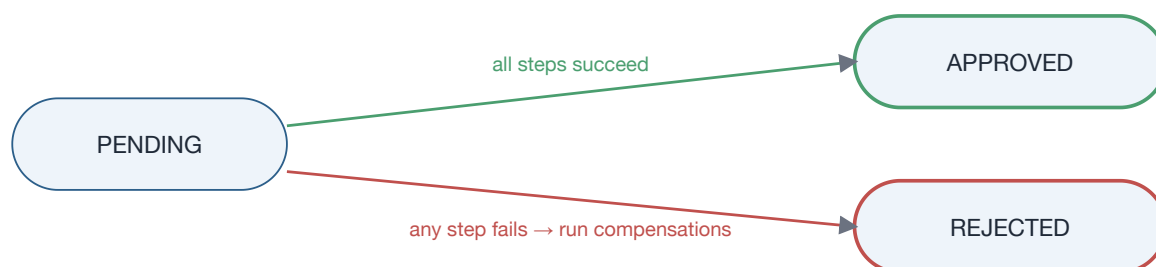


Figure 4.4 — The order's saga-driven state machine: PENDING is the only state a consumer might see while the saga is still running.

Sagas are ACD, not ACID — the missing "I"

A saga gives you **A**tomicity (all steps happen or all are compensated), **C**onsistency, and **D**urability. What it does *not* give you is **I**solation. Between step 1 and step 6, the order sits there in the database in a half-finished, **PENDING** state, visible to any other transaction that happens to read it. A regular database transaction would never expose that half-finished state to anyone else — a saga does, for as long as it takes to run.

ANOMALIES CAUSED BY THE LACK OF ISOLATION

Lost updates: one saga overwrites changes made by another saga, without ever reading them — the second saga has no idea the first one even ran.

Dirty reads: a saga (or anything else) reads data written by another saga that is still mid-flight and might yet be compensated away.

Fuzzy / non-repeatable reads: two steps within the *same* saga read the same record and get different answers, because a different saga changed it in between.

Richardson gives five countermeasures — none of them are magic, they're patterns for containing the damage:

- **Semantic lock:** add an explicit flag that marks a record as "being worked on" — this is exactly what `Order.state = PENDING` and `Ticket.state = CREATE_PENDING` are. Other code that sees the flag knows to treat the record specially (block it, warn about it, or refuse conflicting operations) instead of assuming it's a normal, finished record.
- **Commutative updates:** design updates so that applying them in a different order gives the same result (e.g. credit and debit operations), so it doesn't matter if two sagas' updates interleave.
- **Pessimistic view:** reorder the saga's steps so that the riskiest step — the one most exposed to anomalies — runs as early as possible, shrinking the window where bad data could be read.
- **Reread value:** before overwriting a record, re-read it and check it hasn't changed since you last looked (an optimistic-lock style check); if it changed, abort and restart.
- **Version file:** record every update to a record so out-of-order operations can still be applied correctly regardless of the order messages arrive in.
- **By value:** use the business risk of each specific request to choose the mechanism dynamically — low-value/low-risk operations use a saga, high-value/high-risk ones fall back to a stricter mechanism (even 2PC) for that one request.

Staff-eng lens: "sagas aren't isolated" is the single most common gap in mid-level designs, and semantic locks (a `*_PENDING` status field) are the workhorse fix — cheap, easy to explain to product, and it doubles as the source of truth for the "your order is being processed" UX state. When you present a saga design in a review, walk through what happens if a second request touches the same aggregate while it's **PENDING** — if you don't have an answer, you don't have a finished design.

Designing the Order Service to run a saga

In the orchestration style, `OrderService` doesn't call other services directly. It creates the order, then hands off to the saga orchestration framework (Richardson's example uses Eventuate Tram Sagas) by defining a state machine of steps: which command to send, and which compensating command to send if a later step fails.

```

public class CreateOrderSaga implements SimpleSaga<CreateOrderSagaData> {

    private final SagaDefinition<CreateOrderSagaData> sagaDefinition =
        step()
            .invokeParticipant(this::createOrder)
            .step()
            .invokeParticipant(this::verifyConsumer)
            .step()
            .invokeParticipant(this::createTicket)
            .withCompensation(this::rejectTicket)
            .step()
            .invokeParticipant(this::authorizeCard)
            .step()
            .invokeParticipant(this::approveTicket)
            .step()
            .invokeParticipant(this::approveOrder)
            .build();
}

```

OrderCommandHandlers is the flip side, living inside the participant services (Consumer, Kitchen, Accounting): it listens for command messages from the orchestrator and dispatches them to the real service methods, then replies with success or failure.

```

public class KitchenServiceCommandHandlers {

    public Message createTicket(CommandMessage<CreateTicket> cm) {
        CreateTicket cmd = cm.getCommand();
        Ticket ticket = kitchenService.createTicket(cmd.getRestaurantId(),
                                                    cmd.getTicketDetails());
        return withSuccess(new CreateTicketReply(ticket.getId()));
    }
}

```

Notice what OrderService itself looks like from the outside: a normal service with a normal local database. The saga machinery (the orchestrator, the command handlers, the state field) is what turns a set of independent local transactions into one reliable, resumable business operation.

Pattern: Saga

Problem: How do you maintain data consistency for a business transaction that spans multiple services, each with its own database, without using a distributed ACID transaction?

Solution: Implement the business transaction as a saga: a sequence of local transactions, each in one service, linked by messages or events. If a local transaction fails, execute compensating transactions to semantically undo the local transactions that already completed.

When to use: Any business operation that needs to update data owned by two or more services — which, in a microservices architecture, is most non-trivial operations. Requires the team to accept eventual consistency and design around a lack of isolation.

Pattern: **Choreography-based saga**

Problem: Who decides what step of the saga runs next, without introducing a coordinator every service has to depend on?

Solution: Each participant publishes an event when its local transaction completes; other participants subscribe to the events they care about, run their local transaction, and publish their own event.

When to use: Short sagas with few participants and a simple, mostly-linear flow, where loose coupling matters more than having the whole flow visible in one place.

Pattern: **Orchestration-based saga**

Problem: Choreography's flow gets hard to follow once there are several participants, conditional branches, or multiple compensation paths.

Solution: Define a saga orchestrator (a state machine) that sends command messages telling each participant which local transaction to run, and decides the next step — including compensations — from their reply messages.

When to use: Sagas with several participants, non-trivial branching logic, or where the team needs the whole business flow to be readable, testable, and changeable in one place.

Staff-eng lens: the deepest cost of sagas isn't technical, it's UX and org. Every saga step means a window of time where the record is **PENDING** — product has to design for that (spinners, "we'll email you", disabled buttons) instead of assuming synchronous success/failure. And every new participant in the saga is a new team whose on-call gets paged if their step is slow or their compensation logic has a bug. When you propose "let's add a saga step here," you're also proposing a cross-team operational dependency — say that out loud in the design review, not just the happy-path diagram.

05 Designing business logic in a microservice architecture

This chapter is about how you write the actual business logic *inside* a service, not how services talk to each other. Once you've split a monolith into services (FTGO's Order Service, Kitchen Service, Delivery Service, etc.), each service still has real business rules to enforce. A staff engineer needs to know how to organize that logic so it stays testable and doesn't turn into spaghetti as the service grows.

Two ways to organize business logic

There are two classic styles for writing business logic. Which one you pick depends on how complicated the rules actually are.

Pattern: Transaction Script

Problem: You need to implement a request (a "transaction") and the logic is simple — a few steps, some validation, maybe a database update.

Solution: Write one procedural function ("script") per request. The function does the work top to bottom: load data, validate, compute, save. No fancy object model — just straightforward code.

When to use: Simple domains, CRUD-ish services, or services where the logic genuinely is just a handful of steps. Don't force a rich object model onto simple logic — that's over-engineering.

```
// Transaction Script style
class OrderService {
    void noteRestaurantAcceptedOrder(long orderId) {
        OrderRow order = orderDao.findById(orderId);
        if (!order.getStatus().equals("PENDING"))
            throw new IllegalStateException("bad state");
        orderDao.updateStatus(orderId, "ACCEPTED");
        kitchenClient.notify(order);
    }
}
```

Pattern: Domain Model

Problem: The business logic is complicated — lots of rules, lots of state transitions, lots of edge cases (think FTGO's Order lifecycle: created, approved, rejected, picked up, delivered, cancelled...).

Solution: Use object-oriented design. Each class combines **data** and the **behavior** that operates on that data. An `Order` object doesn't just hold fields — it has methods like `approve()` and `cancel()` that enforce the rules and guarantee the object never ends up in an invalid state.

When to use: Complex domains with many business rules. The extra structure pays for itself because rules live in one place instead of being copy-pasted across scripts.

```
// Domain Model style: behavior lives with the data
class Order {
  private OrderState state;
  void approve() {
    if (state != OrderState.PENDING)
      throw new IllegalStateException("Can't approve: " + state);
    this.state = OrderState.APPROVED;
  }
}
```

ANTI-PATTERN: ANEMIC DOMAIN MODEL

This is what happens when a team says "we're doing OO" but writes classes with only getters/setters and *no* behavior — all the actual logic lives in separate "service" classes. You get the complexity cost of an object model (lots of classes) *without* its main benefit (rules colocated with data). It looks OO, but it behaves like a Transaction Script scattered across too many files. If you see a domain class with 30 getters and zero methods that enforce a rule, that's the smell.

Staff-eng lens: This choice is a judgment call you'll be expected to make on every service review. Don't let anyone tell you "always use Domain Model" — a notifications service with three simple rules doesn't need aggregates and value objects. Match the pattern to the actual complexity of the logic, and be ready to justify the choice in a design doc.

Domain-Driven Design basics: subdomains and bounded contexts

Domain-Driven Design (DDD) is the toolkit behind the Domain Model pattern. Two ideas matter most at the architecture level:

- **Subdomain** — a distinct area of business capability, e.g. "Order Management", "Kitchen Management", "Delivery". DDD says: model each subdomain separately instead of one giant shared model.
- **Bounded context** — the scope inside which a domain model applies and has one consistent meaning. The word "Order" means something slightly different in Order Service (a customer purchase) vs Accounting Service (a line item to bill). A bounded context usually maps to one microservice (or a small group of them) — this is the DDD idea that justifies your service boundaries.

KEY IDEA

Bounded contexts are why microservices avoid a single canonical "God model" of the business. Each service is allowed its own private, simplified view of shared concepts like Order, Consumer, or Restaurant.

The Aggregate pattern

Inside one bounded context you still have multiple classes that belong together. An **aggregate** is a cluster of objects that must be treated as a single unit for the purposes of consistency. One object in the cluster is the **aggregate root** — the only object outside code is allowed to talk to directly.

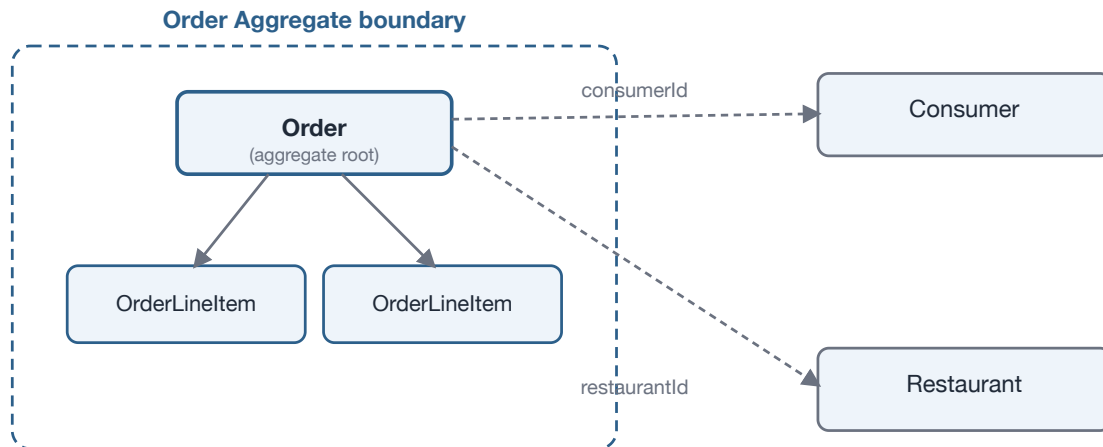


Figure 5.1 — Order aggregate = Order (root) + OrderLineItems, one unit. It refers to Consumer and Restaurant only by ID (dashed), not by object reference — those live in other aggregates/services.

Pattern: Aggregate

Problem: A domain model is a graph of objects. If any code can reach in and mutate any object directly, nothing guarantees the whole graph stays consistent, and it's unclear where transaction boundaries should be.

Solution: Group objects into aggregates. Only the aggregate root is addressable from outside; all changes go through it, so it can enforce invariants (e.g. "total must equal sum of line items"). External code — and other aggregates — refer to it only by ID.

When to use: Any non-trivial Domain Model. It's the unit of consistency, locking, and (as you'll see) the unit of storage in event sourcing.

TWO AGGREGATE RULES THAT MATTER FOR MICROSERVICES

1. Reference other aggregates by identity, not by object reference. An Order holds a restaurantId (a Long/String), never a live Restaurant object. This is what lets Order and Restaurant live in different services with independent lifecycles.

2. A single transaction creates or updates exactly one aggregate. You never update two aggregates atomically in one local transaction. If a use case needs to touch multiple aggregates (possibly in different services), that's exactly the job of a **Saga** — this rule is *why* sagas exist. Aggregates are the pieces sagas coordinate across steps.

WATCH OUT: FUZZY AGGREGATE BOUNDARIES

It's not always obvious what belongs inside an aggregate. Is Restaurant in FTGO one aggregate, or is its menu a separate aggregate? Draw the boundary too big, and you get lock contention and huge transactions (everything serializes through one root). Draw it too small, and you lose the ability to enforce a rule atomically (e.g. "an order can't exceed the restaurant's max daily orders" now needs a saga instead of a simple in-process check).

Staff-eng lens: Aggregate granularity is a real design tradeoff you'll be asked about. Bigger aggregate = easier invariant enforcement, worse concurrency (more contention, bigger locks). Smaller aggregate = better concurrency and independent scaling, but more cross-aggregate coordination via sagas/events. The right answer usually follows the business invariant: "what absolutely must be atomically consistent?" — draw the boundary exactly around that, no more.

Publishing Domain Events

A **domain event** is a small, immutable fact that something meaningful happened to an aggregate — `OrderCreated`, `OrderCancelled`, `DeliveryPickedUp`. Other services and other parts of your own service can react to these instead of polling or calling back synchronously.

Why publish them? Because loose coupling is the whole point of microservices — a domain event lets Order Service announce "this happened" without knowing (or caring) who's listening. Kitchen Service, Accounting Service, and a notification service can all subscribe independently.

- **Identifying events:** look for state transitions in the aggregate's lifecycle — usually one event per meaningful transition (created, approved, shipped, cancelled...).
- **Event enrichment:** include enough data in the event itself (not just an ID) so consumers usually don't need to call back to fetch more — e.g. `OrderCreatedEvent` carries the line items and total, not just an order ID.
- **Generating and publishing reliably:** the update to the aggregate and the publishing of its event must be atomic — you can't update the DB, crash, and lose the event (or publish the event and have the DB write fail). This is exactly the dual-write problem the **Transactional Outbox** pattern (Chapter 3) solves: write the event to an OUTBOX table in the same local transaction as the aggregate update, then a separate relay publishes it to the broker.
- **Consuming events:** subscribers process events from a message channel — typically idempotently, since at-least-once delivery means duplicates happen (more on this in Chapter 6).

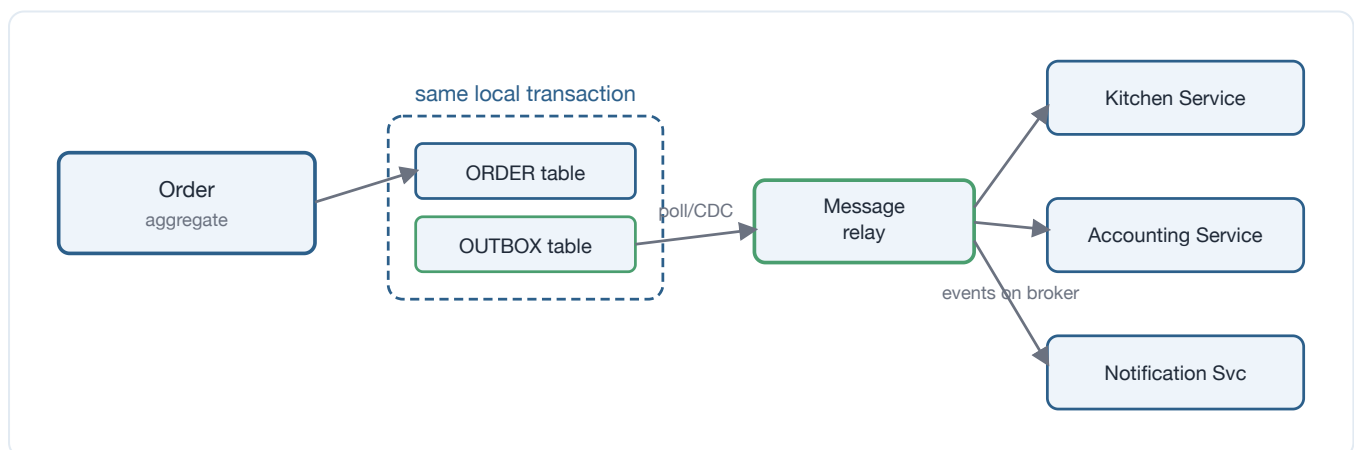


Figure 5.2 — Domain events flow from the aggregate through the transactional outbox, so publishing is atomic with the state change, then a relay pushes them to any number of independent consumers.

Pattern: Domain Event

Problem: Other services (and other parts of the same service) need to know when something important happens to an aggregate, without being tightly coupled to it via synchronous calls.

Solution: The aggregate publishes an immutable event describing the state change. Publish it reliably in the same transaction as the state change (transactional outbox), and let consumers subscribe asynchronously.

When to use: Whenever one service's state change should trigger behavior in another service/aggregate — this is the backbone of choreography-style sagas and of keeping read models up to date.

Chapter 5 showed aggregates publishing events *alongside* saving their current state. This chapter asks: what if events are the *only* thing we store? That's **Event Sourcing** — a different way to persist aggregates that solves several problems from Chapter 5 (like reliably publishing events) as a side effect of how it stores data.

The Event Sourcing pattern

Normally you store the *current state* of an aggregate — one row per Order, and every update overwrites that row. With event sourcing you instead store every *state-changing event* that ever happened to the aggregate, in order, and never overwrite anything. To get the current state, you replay the events from the start (or from a snapshot) and fold them together.



Figure 6.1 — Traditional storage keeps only the latest row (history lost) vs. event sourcing, which appends every event and rebuilds state by replaying the log.

Analogy: think of a bank account. The bank doesn't store just "balance = \$500" and overwrite it on every transaction — it (conceptually) stores the ledger: deposit \$1000, withdraw \$300, withdraw \$200. Your balance is just the sum of the ledger so far. Or think of `git`: the repository isn't "the current files," it's the sequence of commits — you get the current files by replaying commits from the start. Event sourcing does the same thing for an aggregate: current state = replay of all its events.

EXAMPLE: THE ORDER AGGREGATE

Instead of one Order row, the event store has a stream for order-42 containing: `OrderCreated` (line items, restaurant, consumer), `OrderApproved`, `OrderShipped`. To get the Order object, you load the stream and fold: start with an empty Order, apply `OrderCreated` to populate it, apply `OrderApproved` to flip its status, apply `OrderShipped` to flip it again.

```

class Order {
    OrderState state;
    List<OrderLineItem> lineItems;

    void apply(OrderCreatedEvent e) {
        this.lineItems = e.getLineItems();
        this.state = OrderState.CREATED;
    }
    void apply(OrderApprovedEvent e) {
        this.state = OrderState.APPROVED;
    }
}

```

Pattern: Event Sourcing

Problem: Storing only current state loses history, and reliably publishing a domain event alongside a state update needs extra machinery (the outbox from Chapter 5) to avoid the dual-write problem.

Solution: Persist an aggregate as an ordered, append-only sequence of events in an **event store**. Rebuild current state on demand by replaying (folding) the events. Since the events ARE the source of truth, publishing them is no longer a separate write — it's the same write.

When to use: Domains where audit history matters a lot (finance, orders, compliance), or where you're already deep into sagas/domain events and want to eliminate the dual-write problem for good. Not worth it for simple CRUD-ish services.

BENEFITS

- **Reliable audit log for free** — you always know exactly what happened, in order, and why (each event can carry metadata like who triggered it).
- **Temporal queries** — "what did this Order look like last Tuesday?" is just "replay events up to last Tuesday." Impossible with overwrite-in-place storage.
- **Reliably publishes events, automatically** — since the write path already produces events, there's no separate "did the event get lost" problem. This is the event-sourcing answer to the dual-write problem, without needing a separate outbox table.
- **No O/R impedance mismatch on writes** — you're just appending simple event records, not mapping a rich object graph onto normalized tables.

DRAWBACKS / CHALLENGES

- **Learning curve** — it's a genuinely different mental model; teams need time to stop thinking in "current row" terms.
- **Querying is hard** — "find all orders over \$50 that are still pending" means replaying every stream, which is unworkable. The real answer is **CQRS**: build separate, queryable read models kept up to date by subscribing to events (Chapter 7 covers this in depth).
- **Evolving events** — your event schemas change over time; old events in the store must still be readable. You need versioning and "upcasting" (transforming an old event version into the current shape when it's loaded).
- **Deleting data** — an append-only log fights with GDPR's "right to be forgotten," since you can't just delete an old event. Common fix: encrypt sensitive fields per-user and throw away the key when deletion is requested (*crypto-shredding*) instead of rewriting history.
- **Long event streams get slow to replay** — the fix is periodic **snapshots** (a saved current-state checkpoint), so you replay only the events after the snapshot instead of from the beginning.

Staff-eng lens: Event sourcing is a strong "it depends" pattern — bring it up in design reviews as a way to kill the dual-write problem and get free audit history, but be upfront about the cost: you're signing the team up for CQRS-style read models, event versioning discipline, and a genuinely different debugging mental model. If a reviewer asks "why not just use the outbox pattern from Chapter 5 instead," a good answer is: outbox still needs a separate current-state table maintained in lockstep; event sourcing removes that duplication because the event log itself is the only state.

Idempotent handling, the event store as a broker, and concurrency

A few practical mechanics make event sourcing work in production:

- **Idempotent event handling** — consumers may see the same event twice (retries, at-least-once delivery), so handlers must be safe to run more than once (e.g. track processed event IDs, or make the update naturally idempotent).
- **Event store as message broker** — some event stores (the book uses **Eventuate** as an example) double as a publish/subscribe broker: appending an event to a stream both persists it and delivers it to subscribers, so you don't need a separate broker plus outbox.
- **Optimistic concurrency with a version number** — since multiple requests might append to the same aggregate's stream concurrently, each append includes the expected current version; the store rejects the write if the version has moved on, so you can retry instead of silently corrupting history.



Figure 6.2 — The event store doubles as a broker: appended events are pushed to subscribers, including a CQRS view builder that maintains a separate, queryable read model (preview of Chapter 7).

Combining event sourcing with sagas

Recall from Chapter 5 that one transaction may only touch one aggregate — multi-aggregate use cases need a saga. Event sourcing fits naturally with sagas because both are event-driven at heart:

- An event-sourced aggregate naturally emits an event on every state change — that event can be exactly the trigger a saga orchestrator reacts to, no extra plumbing needed.
- You can build an **event-sourcing-based saga orchestrator**: the saga's own state (which step it's on) is itself stored as a sequence of events in an event store, giving the saga the same audit trail and replay benefits as any other aggregate.
- Saga participants that are themselves event-sourced aggregates handle a saga's command by appending an event, which both updates their state and (via the store-as-broker) notifies the orchestrator that the step completed.

Staff-eng lens: This is a nice "everything is consistent" argument to make in a design review: if your aggregates are event-sourced, your sagas can be event-sourced too, and you end up with one uniform mechanism (append event → notify subscriber) for both business logic and cross-service orchestration, instead of maintaining two different plumbing systems (ORM + outbox for aggregates, plus a separate saga state table).

Splitting a monolith into services means splitting the database too. Each service now owns its own private data. That's great for independent deployability, but it breaks something you took for granted in the monolith: the humble SQL JOIN. This chapter is about how to answer questions that span multiple services' data without gluing everything back into one giant database.

The problem: no cross-service JOIN

Imagine a support agent wants to see "Order #42, with the restaurant's name and address, the delivery status, and the payment status" — all on one screen. In the monolith, that was one SQL query with three JOINS. In FTGO, the **Order**, **Restaurant**, **Delivery**, and **Accounting** services each guard their own private database. Nobody outside a service is allowed to reach into its tables directly.

Analogy: it's like four separate government departments each keeping their own paper files on you. If the DMV wants your tax history, it can't just walk into the IRS building and pull a drawer open — it has to ask, or someone has to have already copied that data over in advance.

KEY IDEA

There are two honest ways to answer a query that spans services: ask every owning service at query time and stitch the answers together (**API Composition**), or keep a pre-joined, pre-stitched copy of the answer ready in advance (**CQRS**).

Pattern: API Composition

An **API composer** — could be the API gateway, could be a dedicated service — receives the query, calls each service that owns a piece of the answer, and joins the results in memory before returning one response to the caller.

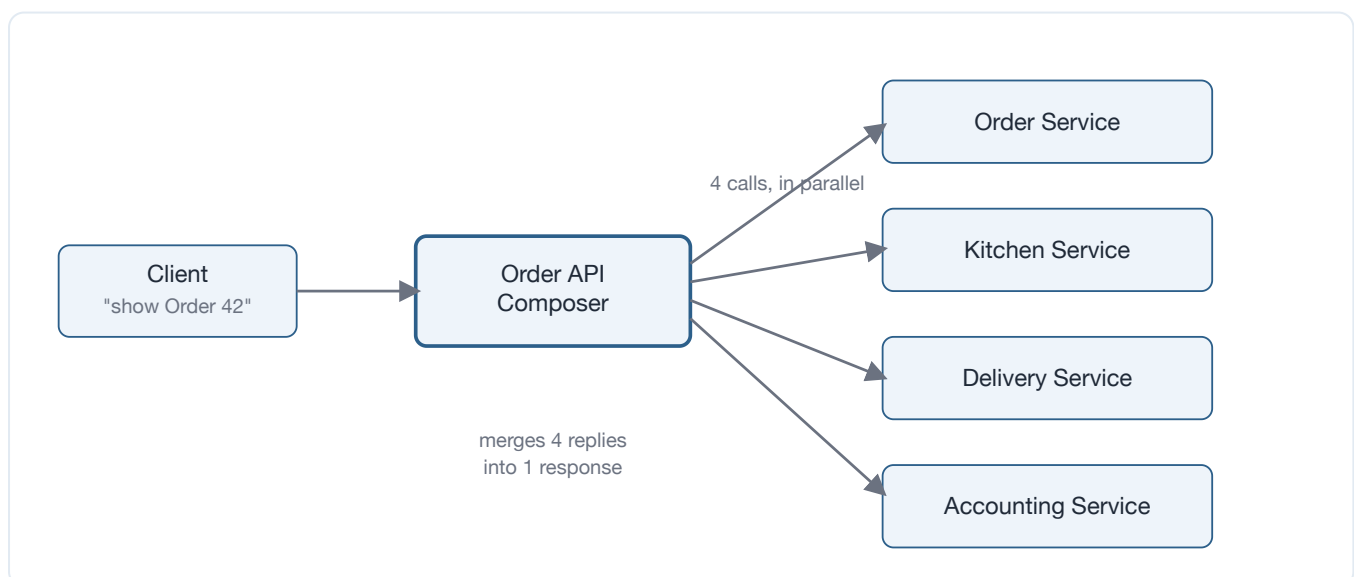


Figure 7.1 — API composer fans out to four services and joins the results in memory.

TRADEOFFS / WATCH OUT

Availability drops: the composed query is only as available as the least-available service it calls. Four services at 99.9% uptime each combine to less than 99.9% overall.

Inefficient joins: joining in application memory is not as efficient as a database engine doing it, especially for large result sets.

Overfetching: a service may have to return more fields than needed just so the composer can pick out the ones it wants.

Pattern: API Composition

Problem: a query needs data owned by multiple services.

Solution: an API composer invokes the owning services and combines the results in memory.

When to use: simple queries, low-to-moderate volume, where slightly lower availability and in-memory joining are acceptable.

Pattern: CQRS (Command Query Responsibility Segregation)

When API composition isn't good enough — the query is complex, needs a different kind of database (full-text search, geospatial), or is run very often — reach for **CQRS**. The idea: split your data model into a **write model** (handles commands, enforces business rules — this is your normal services and their DBs) and one or more **read models** (handles queries only, pre-joined and shaped exactly for the question being asked).

The read model is a denormalized **view database**. It's kept up to date by a **view updater** that subscribes to the domain events published by the write-side services (order created, delivery picked up, payment authorized, etc.) and updates the view accordingly. Queries never touch the write-side services at all — they go straight to the view.

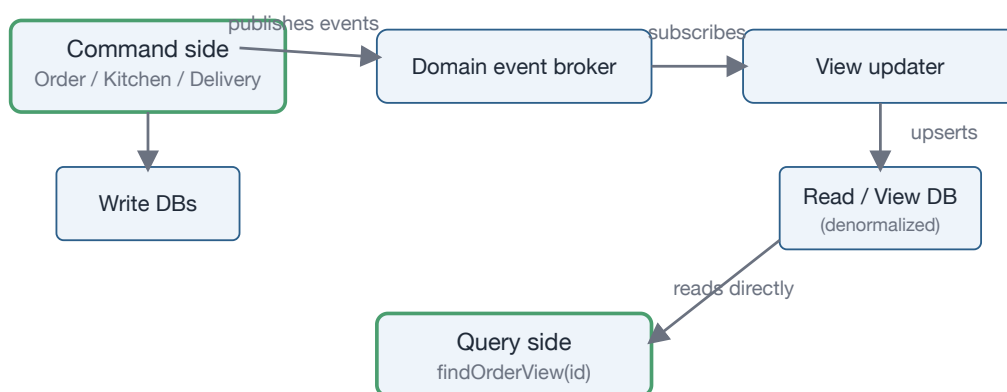


Figure 7.2 — CQRS: write side publishes events; a view updater keeps a denormalized read DB current; queries hit the read DB directly.

Analogy: think of a newsroom's live results board during an election. Each precinct (service) reports its own tally as it happens (event). A dedicated clerk (view updater) copies every incoming number onto one

big board (view DB) that's already laid out the way viewers want to see it. Nobody reading the board calls the precincts directly.

BENEFITS

Queries are fast and efficient — no runtime joins, no fan-out calls. You can use the best datastore for the job, e.g. Elasticsearch for text search, even if the write side uses relational DBs. It pairs naturally with **event sourcing**, since you already have an event stream to subscribe to. Read and write sides can scale independently.

TRADEOFFS / WATCH OUT

More moving parts: now you maintain view updaters and extra databases. The read model lags behind the write model — this is **eventual consistency** / replication lag, so a user can occasionally see stale data right after making a change. View updaters must handle events that arrive **out of order** or more than once, so updates need to be **idempotent** (applying the same event twice must not corrupt the view).

Pattern: CQRS

Problem: queries need data from multiple services, need a specialized query datastore, or must be fast and high-volume; API composition is too slow, too fragile, or too limited.

Solution: maintain one or more denormalized read-only view databases, kept current by subscribing to domain events, and query them directly.

When to use: complex or high-volume queries, need for specialized read datastores (search, analytics), or already using event sourcing.

Staff-eng lens: API composition vs CQRS is really a consistency-vs-simplicity tradeoff, the same tension you'll defend in design reviews again and again. API composition keeps you strongly consistent (you always read live data) at the cost of availability and latency. CQRS buys you speed and flexibility but forces you to answer the hard question "how stale can this screen be, and who gets hurt if it is?" Staff engineers are expected to make that staleness budget explicit — in SLAs, in UX copy ("processing...") — rather than let it become an unspoken bug report.

Splitting FTGO into services solved the monolith's problems, but it created a new one at the edge: how do mobile apps, web apps, and third-party partners talk to a system made of dozens of independently deployed services? This chapter is about designing the front door.

The problem: too many doors

If every client calls services directly, you get a mess: clients become **chatty** (many round trips for one screen), tightly **coupled** to internal service boundaries that you'd like to be free to change, forced to speak whatever protocol each service happens to use, and stuck doing their own composition and retries. It also leaks internal architecture to the outside world — rename or split a service, and now external partners break.

Analogy: imagine a hospital with no reception desk — visitors wandering the halls trying to find radiology, billing, and the pharmacy themselves, each department speaking a different language. A reception desk (gateway) routes you, translates, and shields you from knowing the hospital's internal floor plan.

Pattern: API Gateway

An **API Gateway** is a single entry point that sits between clients and the internal services. It routes requests to the right service(s), can do **API composition** itself, translates protocols (say, HTTP/JSON outside, gRPC inside), and handles cross-cutting "edge" concerns once instead of in every service: authentication, rate limiting, response caching, metrics, and logging. Real examples: Netflix Zuul, Spring Cloud Gateway.

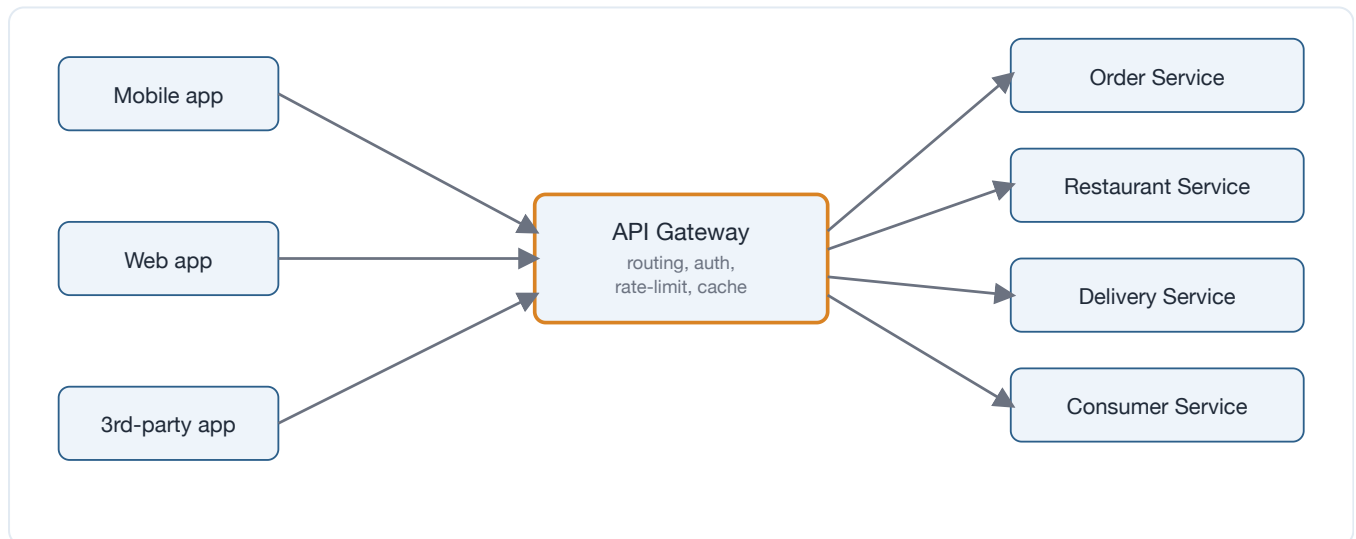


Figure 8.1 — One API Gateway fronts every client and routes into the internal services.

TRADEOFFS / WATCH OUT

A single shared gateway can turn into a **bottleneck** — every request in the system flows through it, so its capacity and latency now cap the whole platform. It's also a **single point of failure**: if it goes down, every client goes down, even if all backend services are healthy. And because every team's routes live in one gateway config, it easily becomes an **organizational bottleneck** too — one team owns a resource that every other team must change to ship a feature.

Pattern: Backends For Frontends (BFF)

Instead of one gateway trying to serve mobile, web, and partners with one-size-fits-all endpoints, give **each client type its own gateway**, owned by the team that owns that client. The mobile team owns the mobile BFF and shapes its responses exactly for the mobile app's screens; the web team owns the web BFF; a public BFF (with stricter auth and rate limits) serves third parties.

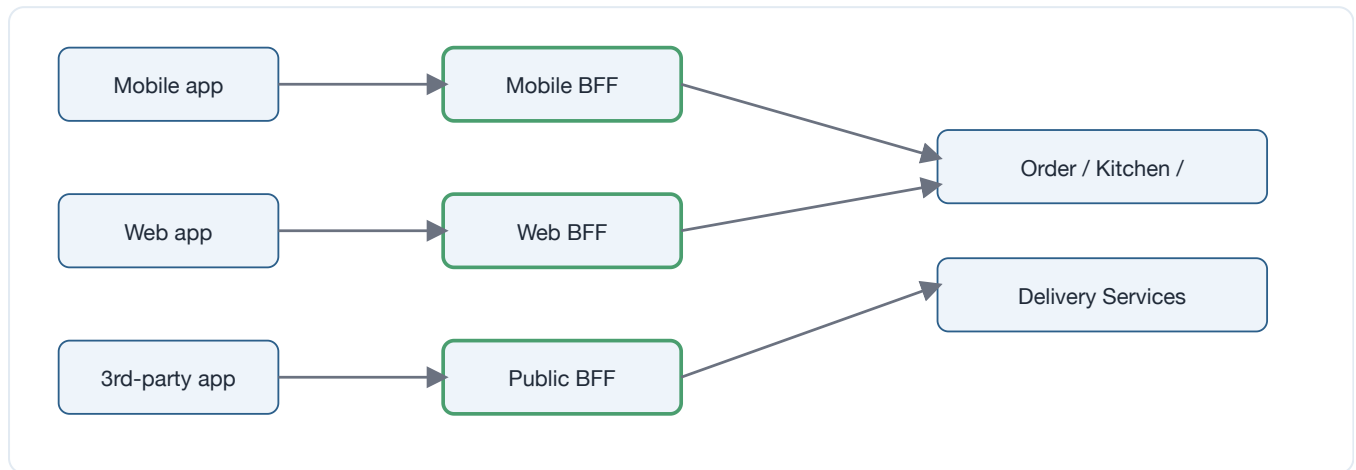


Figure 8.2 — Backends For Frontends: each client type gets its own gateway, owned by that client's team.

BENEFITS

Each BFF is small, owned end-to-end by one team, and can evolve at that team's pace without coordinating with every other client team. No more one giant shared gateway codebase that everyone is scared to touch.

TRADEOFFS / WATCH OUT

Cross-cutting edge concerns (auth, rate limiting, TLS) now need to be solved once **per BFF**, usually by putting them in a shared library or a thin infrastructure layer in front of all the BFFs — otherwise you just re-created the single-gateway bottleneck problem, duplicated three times.

Pattern: API Gateway

Problem: clients need a simple, stable way to talk to a system built from many independently evolving services.

Solution: a single entry-point service that routes requests, composes results, translates protocols, and handles cross-cutting edge concerns.

When to use: you have multiple client types and want to hide internal service structure; watch it for becoming a bottleneck.

Pattern: Backends For Frontends (BFF)

Problem: one shared gateway serving very different clients (mobile, web, partners) turns into contention between teams and a one-size-fits-none API.

Solution: give each client type its own gateway, owned by that client's team.

When to use: distinct client types with different data/latency needs and separate owning teams.

Implementation concerns

A gateway sits on the critical path of every request, so its own implementation matters a lot:

- **Performance:** use a reactive, non-blocking I/O model (e.g. Netty-based frameworks) so one gateway instance can hold open many slow, concurrent client connections without burning a thread per request.
- **Partial failure:** wrap calls to backend services in **circuit breakers** so one slow or dead service doesn't stall or crash the gateway for everyone.
- **Service discovery:** the gateway needs to find the current network location of each service instance, the same discovery mechanism used service-to-service.
- **GraphQL** is a popular way to build a flexible gateway/BFF: clients ask for exactly the fields they need in one request, and the gateway resolves each field from the owning service, cutting down on both overfetching and chattiness.

```
// Simplified circuit breaker around a downstream call from the gateway
CircuitBreaker cb = CircuitBreaker.of("kitchenService", config);
Mono<KitchenStatus> status = cb.run(
    kitchenServiceClient.getStatus(orderId),
    throwable -> Mono.just(KitchenStatus.UNKNOWN) // fallback
);
```

Staff-eng lens: "API gateway" is often the first answer junior designs reach for, but a staff engineer's job is to name the cost out loud: it's a shared, latency-critical, blast-radius-maximizing piece of infrastructure. The BFF pattern is really an org-design decision disguised as an architecture pattern — it moves a coordination bottleneck (one gateway, many teams) into parallel ownership (many gateways, one team each), at the price of duplicating edge-concern plumbing. When you're asked "should we have one gateway or a BFF per client," the right answer starts with "how many client teams do we have, and how often do they collide today" — not with the technology.

A microservices app is a web of small services calling each other over the network. That makes testing harder than in a monolith: you cannot just spin up "the app" — you have to decide which parts to test together, which to fake, and how to catch breakages between services without waiting hours for a giant end-to-end test suite. This chapter is about the strategy: the test pyramid, test doubles, and the star trick — consumer-driven contract tests — that catches integration bugs cheaply.

Why testing microservices is hard

In a monolith, one process holds all the logic, so an in-process test can exercise most of it. In FTGO, placing an order touches the **Order Service**, **Kitchen Service**, **Restaurant Service**, **Consumer Service**, **Accounting Service**, and **Delivery Service** — six separately deployed processes talking over REST and messaging. A bug can hide in any single service, or in the contract between two services (for example, Order Service expects a field that Kitchen Service stopped sending).

Analogy: Testing a monolith is like checking one large machine on a bench. Testing microservices is like checking a fleet of delivery trucks that also need to talk to each other by radio — you must test each truck alone, and also test that the radios agree on the same language.

WATCH OUT

The naive fix — "just run everything together and click through the app" (end-to-end testing) — works, but it is slow, flaky, and only tells you something is broken, not what or where. It does not scale as the number of services grows.

The Test Pyramid

The **Test Pyramid** is a budget for where to spend your testing effort. Write lots of tiny, fast tests at the bottom, and fewer, slower, broader tests as you go up.

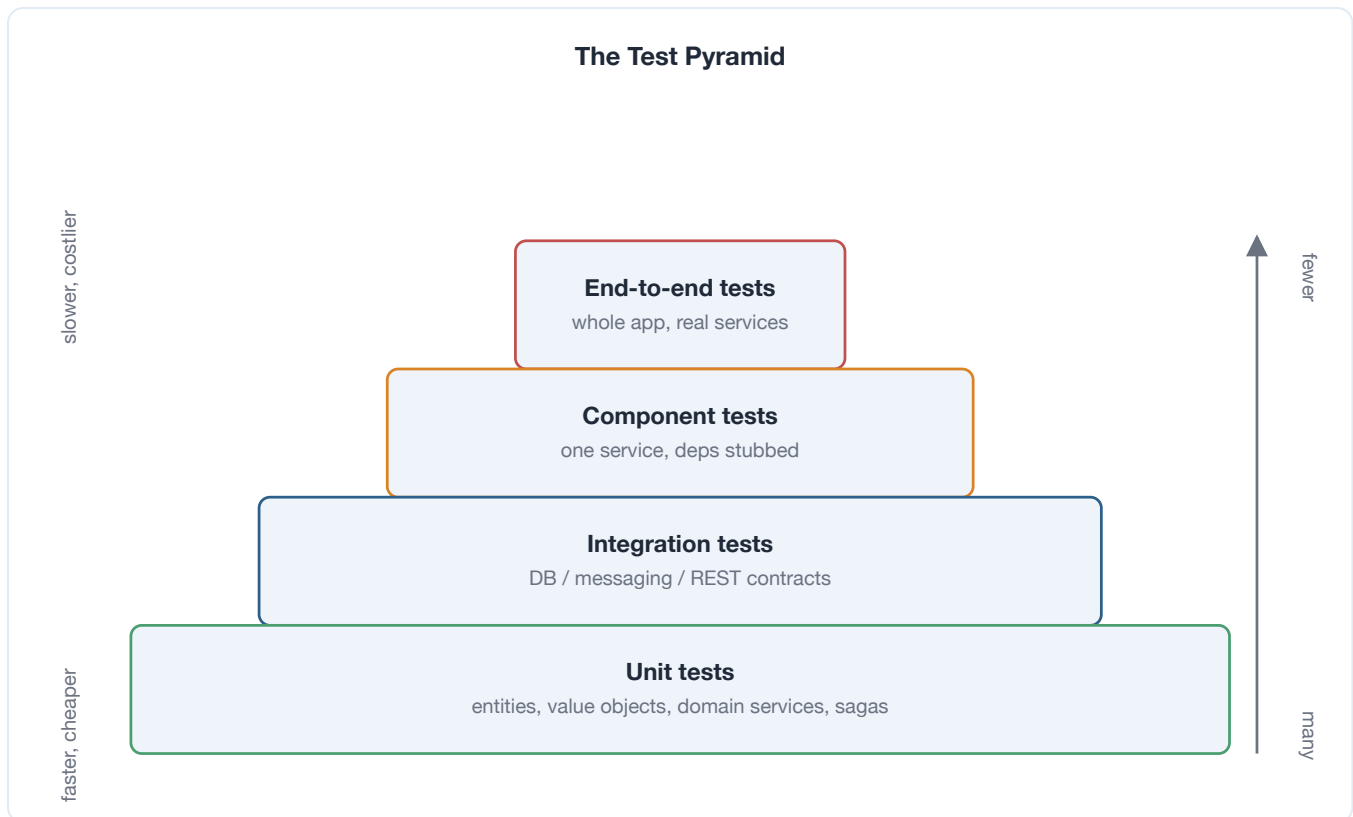


Figure 9.1 — The test pyramid: wide, fast, cheap unit tests at the base; a narrow, slow, expensive sliver of end-to-end tests at the top.

KEY IDEA

Push tests as far down the pyramid as you can. A bug caught by a unit test costs milliseconds and no infrastructure. The same bug caught only by an end-to-end test costs minutes, a full environment, and a flaky-test debugging session.

Staff-eng lens: The pyramid shape is a cost curve, not a rulebook. When you review a team's test suite, check the shape: if it looks like an ice-cream cone (mostly slow E2E tests, few unit tests), CI will be slow and flaky, and people will start skipping tests before merging — a classic organizational failure mode you should flag early.

Test doubles: stubs and mocks

A **test double** is a fake stand-in for a real dependency, so a test does not need the real thing running.

- **Stub:** returns canned answers. You use it to feed input into the code under test. Example: a stub Kitchen Service client that always returns "ticket accepted."
- **Mock:** like a stub, but the test also verifies it was called correctly — the right method, the right arguments, the right number of times. Example: verifying Order Service called `notifyAccepted()` exactly once.

EXAMPLE

```
// stub: just returns canned data
when(kitchenClient.getTicket(id)).thenReturn(ticket);

// mock: also verifies the call happened
verify(kitchenClient).acceptTicket(orderId);
```

Consumer-driven contract testing

Two services agree on an API. If the provider (say, Kitchen Service) changes its response shape, the consumer (Order Service) can break — but you would not know until a slow end-to-end test fails, or worse, until production. **Consumer-driven contract testing** fixes this: the consumer writes down exactly what it expects, and that expectation becomes an automated test that runs against the provider's real code, on every build.

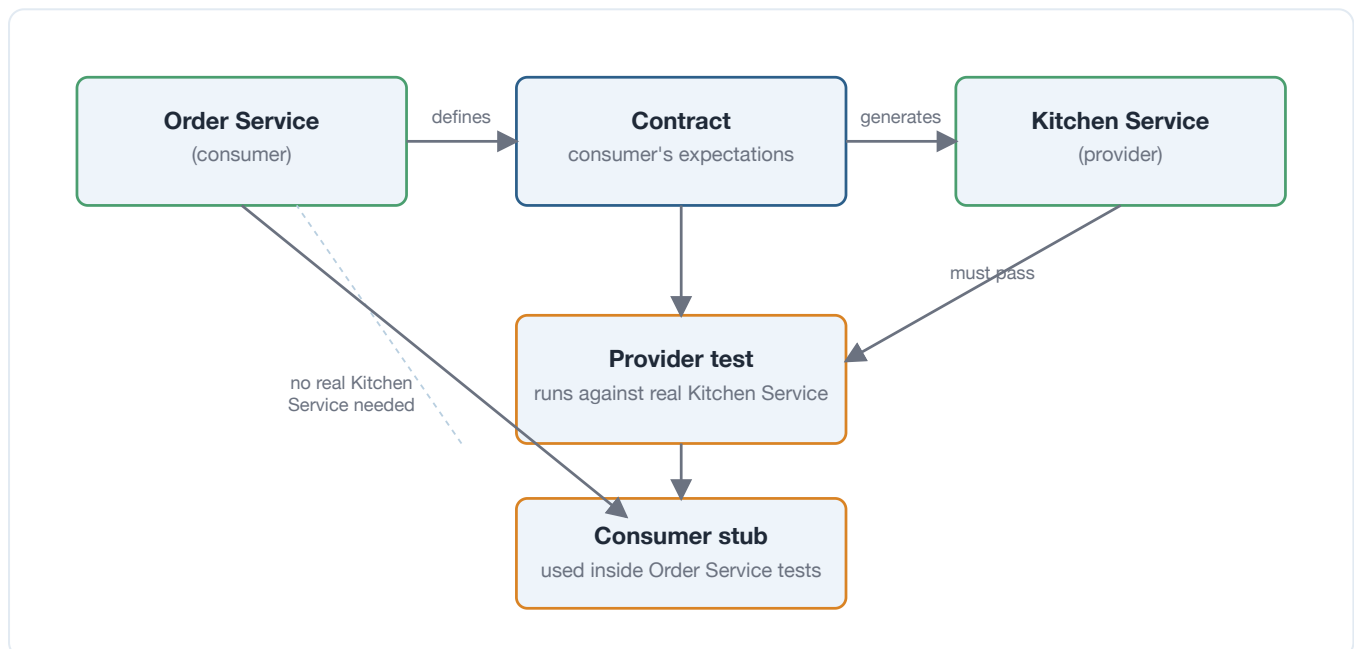


Figure 9.2 — Consumer-driven contract testing: the consumer's contract both generates a provider test (verifies Kitchen Service honors it) and a stub the consumer uses instead of the real service.

With tools like **Spring Cloud Contract**, the contract file (JSON or Groovy) is the single source of truth. It produces two artifacts:

- **Provider-side contract test:** runs in Kitchen Service's build, replays the contract's requests against real Kitchen Service code, and fails the build if the response no longer matches. This is how the provider learns it broke a consumer, immediately, without waiting for Order Service to even exist in the pipeline.
- **Consumer-side contract test:** Order Service tests run against a generated stub of Kitchen Service (built from the same contract), so Order Service can verify its own code works against that expected shape — no network call to a real Kitchen Service needed.

BENEFITS

Contract tests run in seconds, live inside each service's own build, and catch the exact class of bug (API drift between services) that end-to-end tests exist for — but far cheaper and without needing every service running at once.

Staff-eng lens: This is the pattern to bring up when someone proposes "more end-to-end tests" as the fix for cross-service breakage. Contract tests give the same safety net at a fraction of the cost and flakiness, and — importantly — they put the responsibility for the interface in the right place: the provider team runs a test written from the consumer's actual needs, so they cannot silently break a client they don't own.

Pattern: Consumer-driven contract test

Problem: A provider service can break a consumer without knowing it, and full end-to-end tests to catch this are slow and flaky.

Solution: The consumer specifies its expectations of the provider's API as a contract; this contract becomes a test that runs in the provider's build.

When to use: Whenever two services communicate over REST or messaging and are built/deployed independently — which, in microservices, is always.

Pattern: Consumer-side contract test

Problem: Testing a service's client code against a real remote service is slow and requires that service to be running and reachable.

Solution: Test the consumer's code against a stub generated from the same contract used to verify the provider, so both sides stay in sync.

When to use: For any service that calls another service, as part of that service's own fast test suite.

Unit testing business logic

Most of the pyramid's base is unit tests of the domain model: entities (like an `Order` aggregate), value objects, domain services, and sagas. These need no network, no database, no other service — just the object and its inputs.

```
@Test
void shouldRejectOrderWhenTotalBelowMinimum() {
    Order order = new Order(items, restaurant);
    assertThrows(OrderMinNotMetException.class,
        () -> order.verifyDeliveryInformation());
}
```

Sagas (multi-step, multi-service transactions like "Create Order") can also be unit tested in isolation by feeding them a sequence of reply messages and asserting which commands they send next — no real services required.

KEY IDEA: THE DEPLOYMENT PIPELINE

All these test types are stitched together into a **deployment pipeline**: commit triggers unit tests, then integration tests, then component tests, then (rarely) a small end-to-end suite, before a build is promoted toward production. Fast, cheap tests run first and fail fast; slow tests run last and only when earlier stages passed.

Chapter 9 covered unit tests and contracts. This chapter climbs the rest of the pyramid: integration tests (does my service really talk to its database and neighbors correctly?), component tests (does this one service behave correctly as a whole, with everything else faked?), and end-to-end tests (does the whole system work for a real user journey?) — while explaining why you want as few of the last kind as possible.

Integration testing

Integration tests verify that a service's code correctly talks to things it does not control: its database, a message broker, or another service's REST API. The trick is to test the edges without needing the whole system running.

- **Persistence tests:** run the service's repository code against a real (or in-memory / containerized) database to check the ORM mappings and queries actually work.
- **Messaging contract tests:** verify a service publishes/consumes messages in the format its partners expect — using the same consumer-driven contract idea as REST.
- **REST contract tests:** verify a service's REST client/controller matches the agreed contract, same as above.

TRADEOFFS

Persistence tests need real infrastructure (e.g. a test database via Testcontainers), so they are slower than pure unit tests — but far faster and more reliable than spinning up the whole FTGO system.

Component testing

A **component test** tests one service — say, Order Service — as a whole black box, through its real API, but with every one of its external dependencies replaced by a stub: its database, Kitchen Service, the message broker, all faked. You get confidence the service works end-to-end internally, at a fraction of the cost of a real end-to-end test.

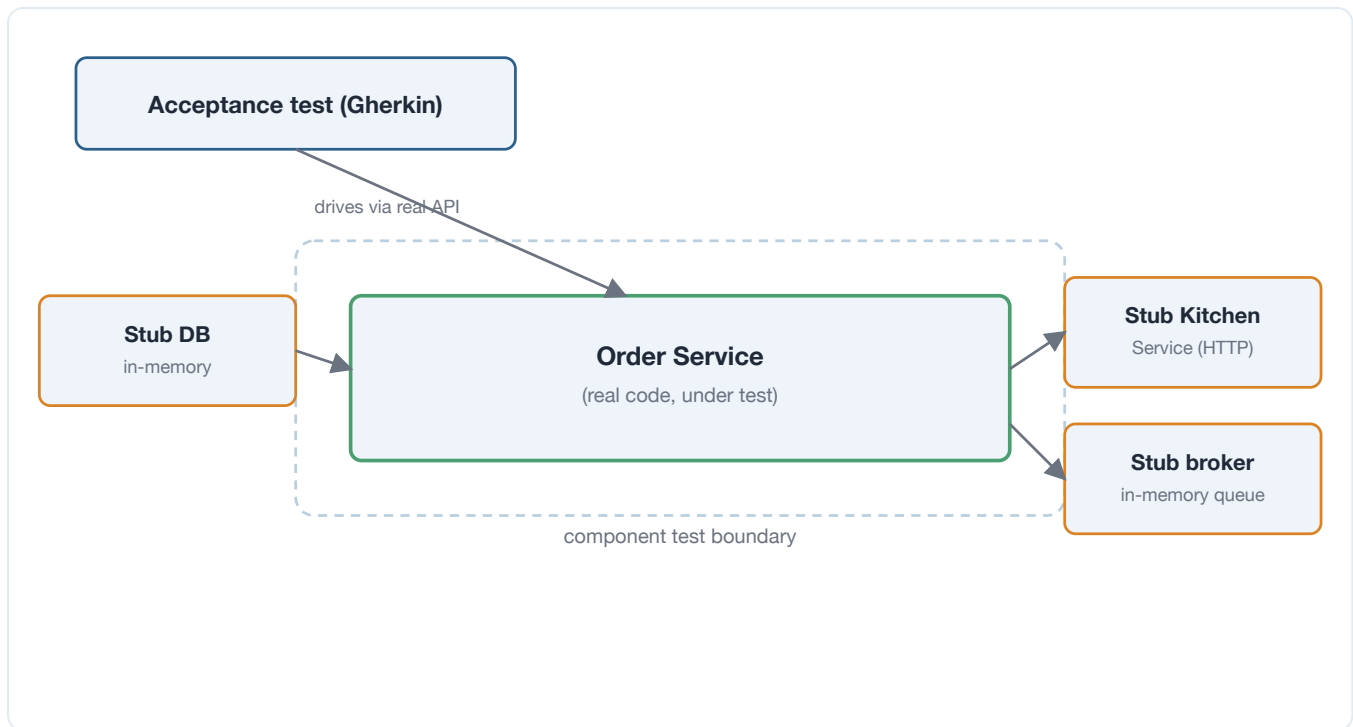


Figure 10.1 — Component test: *Order Service* runs with its real code, driven through its real API, while the database, Kitchen Service, and message broker are all stubbed.

Stubs can be **in-process** (fake objects wired in via dependency injection, running in the same JVM — fast, but skips the network layer) or **out-of-process** (a small stub HTTP server or in-memory broker, closer to real deployment — slightly slower, more realistic).

Acceptance criteria are often written in **Gherkin** and run with **Cucumber**, so a business-readable scenario becomes the actual test:

```
Scenario: Accept a valid order
  Given a consumer with a valid credit card
  When the consumer places an order for $25
  Then the order should be ACCEPTED
```

Pattern: **Service component test**

Problem: You want confidence a whole service works correctly, but running the real dependencies (other services, brokers) makes tests slow, flaky, and hard to set up.

Solution: Test the service in isolation, through its real API, with all external dependencies replaced by stubs.

When to use: For every service, as the main way to verify its behavior as a whole — before reaching for end-to-end tests.

Analogy: A component test is like testing one truck's engine, brakes, and dashboard on a stand with fake road resistance and a fake radio operator — you learn a lot about that truck without needing the whole highway of other trucks running.

End-to-end testing

End-to-end tests run the entire FTGO system — every service, real databases, real message brokers — and drive it through the UI or top-level API, exactly like a real user placing an order.

WATCH OUT

End-to-end tests are slow (minutes, not milliseconds), expensive to set up and maintain, and flaky — a timing hiccup in any one of a dozen services can fail the whole test for no real reason. Debugging a failure means hunting across every service's logs.

Because of this cost, the rule is: keep the end-to-end suite tiny, and use it only for the handful of key user journeys that really matter end to end — for example, "a consumer places an order, the restaurant accepts it, and delivery completes" — not for every edge case (those belong lower in the pyramid).

Staff-eng lens: When you inherit a system with a bloated, flaky end-to-end suite, the fix is rarely "make the tests less flaky" — it is usually "most of these shouldn't be end-to-end tests at all." Push logic checks down to unit tests, cross-service checks to contract tests, and per-service behavior to component tests. Reserve E2E for the two or three journeys whose failure means the business is actually broken. This reasoning — matching test cost to the risk it mitigates — is exactly the kind of tradeoff judgment a staff engineer is expected to bring to a testing strategy review.

Splitting a monolith into microservices is only half the job. Every one of those services now needs to handle security, configuration, and "can I see what's happening inside you right now?" on its own. This chapter is about the plumbing that makes services safe to run in production — and why, at staff level, you'll spend more time defending these patterns in design reviews than writing the business logic itself.

Security: who is calling, and what are they allowed to do?

In a monolith, you log in once and the app keeps your identity in a session, in memory, on one machine. In FTGO, a request from the Consumer Service's mobile app might touch the Order Service, then the Kitchen Service, then the Delivery Service. Each of those is a separate process, maybe on a separate machine. They all need to know: who is this user, and what are they allowed to do?

KEY IDEA

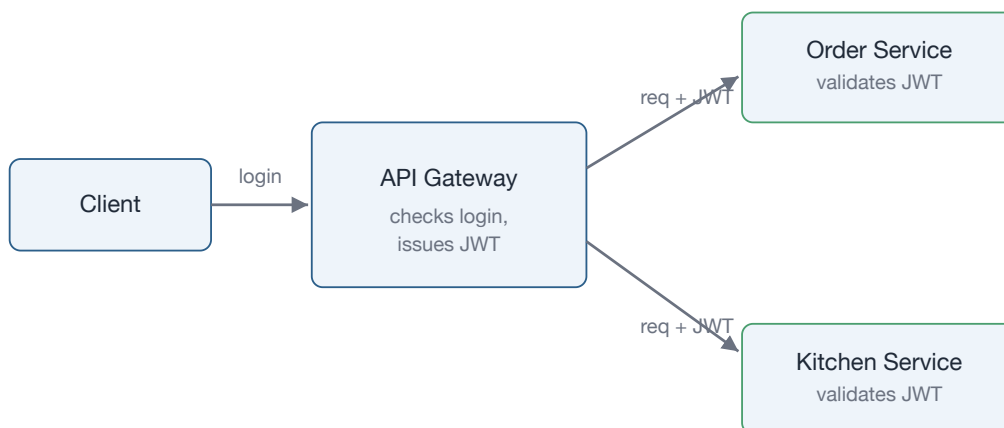
Authenticate once, at the edge (the API gateway). Don't make every downstream service re-check a password. Instead, pass a signed, tamper-proof token — the **access token** — downstream, and let each service verify and trust it.

Pattern: Access token

Problem: After the API gateway authenticates a request, how does the identity (who) and authorization (what they can do) information flow to the services that actually handle the request?

Solution: The API gateway attaches a signed token (usually a **JWT** — JSON Web Token) to each downstream request. The token carries the user's identity plus roles/scopes. Each service verifies the token's signature and reads the claims instead of calling a central auth service on every request.

When to use: Almost always, in any microservice architecture with an API gateway. It avoids a login-check network call fan-out on every hop.



Gateway authenticates once; JWT carries identity + roles/scopes to every downstream hop.

Figure 11.1 — Access token flow: gateway authenticates, JWT travels downstream, each service validates it independently.

The JWT is just a signed JSON blob: something like `{"sub":"consumer-42","roles":["CONSUMER"]}`, signed with a key the services can verify (either a shared secret or the auth server's public key). No service needs to call back to a login database — it just checks the signature and reads the claims.

OAuth 2.0, in plain terms

OAuth 2.0 is the standard playbook for issuing these tokens. The pieces:

- **Authorization server** — the one place that verifies a username/password (or client credentials) and mints tokens. Think of it as FTGO's identity department.
- **Access token** — short-lived (minutes), sent with every request, says "this bearer is allowed to do X."
- **Refresh token** — long-lived, kept safe, used to get a new access token without asking the user to log in again.
- **Scopes** — fine-grained permissions baked into the token, e.g. `order:read`, `order:write`.

Analogy: The access token is like a concert wristband. Security at the front gate (the authorization server) checks your ticket once and puts a wristband on you. Every stage entrance (each service) just glances at the wristband — nobody re-checks your ticket. The wristband expires at midnight (short-lived access token); if you want back in tomorrow, you need a new one, but you didn't have to keep your ticket stub for that (refresh token handles renewal).

Authorization: roles and ACLs

Authentication answers "who are you?" **Authorization** answers "what are you allowed to do?" Two common approaches:

- **Role-based (RBAC):** a user has roles like `CONSUMER`, `RESTAURANT_STAFF`, `COURIER`, and each role is allowed certain operations. Simple, scales well, is what most services use.
- **ACLs (access control lists):** per-resource permission lists — "consumer 42 can view order 917." More precise, more overhead. Useful when access depends on ownership of a specific record, not just a role.

Staff-eng lens: In interviews and design docs, the detail people forget is token revocation. JWTs are self-contained and stateless — great for scale, terrible if you need to instantly kill a compromised session, because nobody "checks back in" with the authorization server. The usual mitigation is short expiry (minutes) plus refresh tokens that are checked centrally and can be revoked. Know this tradeoff; it's a classic follow-up question.

Externalized Configuration

Every service needs config that differs by environment: database URL, credentials, feature flags, timeouts. Hardcoding any of it means rebuilding the container just to change a password — a staff-eng smell.

Pattern: Externalized configuration

Problem: A service's configuration (DB credentials, endpoints, flags) varies by environment (dev/staging/prod) and must never be baked into the code or image.

Solution: Supply configuration from outside the application at deploy time or startup/runtime, and read it through a uniform API.

When to use: Always — the only question is push vs pull.

Two ways to deliver that config:

- **Push model:** the deployment environment hands config to the service — environment variables, a mounted properties file, container orchestrator secrets/config maps. Simple, works everywhere, but requires a restart/redeploy to change values.
- **Pull model:** the service asks a **config server** for its configuration at startup (or even live, at runtime) — e.g. Spring Cloud Config Server, HashiCorp Vault, Consul, ZooKeeper. This enables dynamic updates without a redeploy, and centralizes secrets management (rotation, access auditing, encryption at rest).

EXAMPLE

The Order Service's database password shouldn't live in a YAML file in Git. With the pull model, it starts up, authenticates to Vault with a short-lived identity (e.g. a Kubernetes service account token), and pulls the current DB credential — which Vault can rotate without anyone touching the Order Service's deployment.

Staff-eng lens: Push is simpler and has fewer moving parts (no config server to keep available), but every change is a redeploy. Pull enables live updates and centralized secret rotation, but now the config server is a dependency every service needs at startup — if it's down, nothing boots. Plan its availability like a tier-1 system.

Observability patterns

This is the part staff engineers get graded on hardest: when something breaks in production across a dozen services, can you find out why in minutes, not hours? Microservices trade "one process to debug" for "a distributed system to observe." These patterns are how you buy that visibility back.

Pattern: Health check API

Problem: The deployment platform (or load balancer) needs to know if a service instance is alive and ready to take traffic, so it can restart it or stop routing to it.

Solution: Expose an endpoint, e.g. **GET /health**, that checks the service and its critical dependencies (DB connection, message broker) and returns healthy/unhealthy. Distinguish **liveness** (is the process alive at all?) from **readiness** (is it ready to serve traffic, e.g. has it finished warming up its cache?).

When to use: Every service, no exceptions — it's what Kubernetes probes or an ELB health check polls.

Pattern: Log aggregation

Problem: Logs are scattered across dozens of service instances on dozens of hosts. SSH-ing into each one to grep logs doesn't scale.

Solution: Ship every service's logs to a central, searchable store — the classic stack is Elasticsearch (storage/search), Logstash/Fluentd (collection), Kibana (UI), often called **ELK**.

When to use: Always, from day one. Retrofitting log aggregation during an incident is too late.

Pattern: Distributed tracing

Problem: One user request fans out across several services. A log line in the Kitchen Service doesn't say which Order Service request caused it.

Solution: Assign every incoming request a unique **trace id** (a.k.a. correlation id) at the edge. Pass it along on every downstream call (headers). Each service records a **span** — a timed unit of work — tagged with that trace id, and reports spans to a tracing server (Zipkin, Jaeger) that stitches them into one timeline. The **OpenTracing** API standardized how libraries instrument this.

When to use: Any system with more than a couple of services calling each other. It's how you answer "why was this request slow?" without guessing.

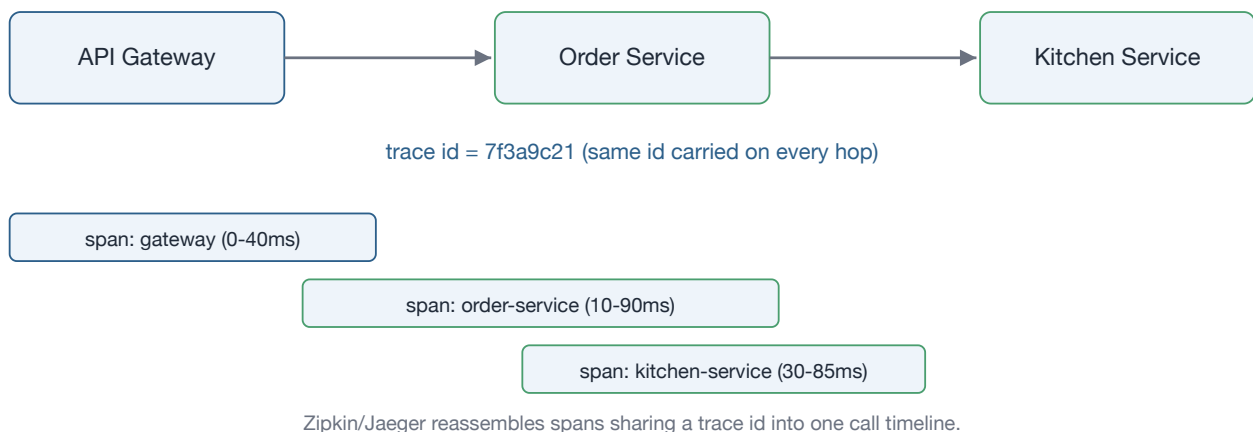


Figure 11.2 — Distributed tracing: one trace id, spans across three services, reassembled into a timeline.

Staff-eng lens: Correlation IDs are non-negotiable, not a nice-to-have. Without one, an incident review means matching timestamps by hand across five log streams and guessing. With one, it's a single query. If you review a design doc for a new service and it doesn't propagate a trace/correlation id on every outbound call, that's a blocking comment — this is the cheapest reliability investment you can make.

Pattern: Application metrics

Problem: You need numeric, aggregatable signals (request rate, error rate, latency, queue depth) to know if a service is healthy and to alert before things fully break.

Solution: Instrument the app to record metrics (e.g. via Micrometer as a facade) and expose or ship them to a metrics server such as Prometheus. Two delivery styles: **push** (service actively sends metrics to a collector, good behind firewalls / short-lived jobs) vs **pull/scrape** (the metrics server periodically hits a `/metrics` endpoint — Prometheus's default, simpler service-side).

When to use: Every service — this is what your dashboards and alerts are built on.

Pattern: Exception tracking

Problem: Buried in logs, thousands of duplicate exceptions drown out the one new, important one.

Solution: Send exceptions to a dedicated tracker that deduplicates identical stack traces, counts occurrences, and can alert only on new or spiking exception types (Sentry-style tools).

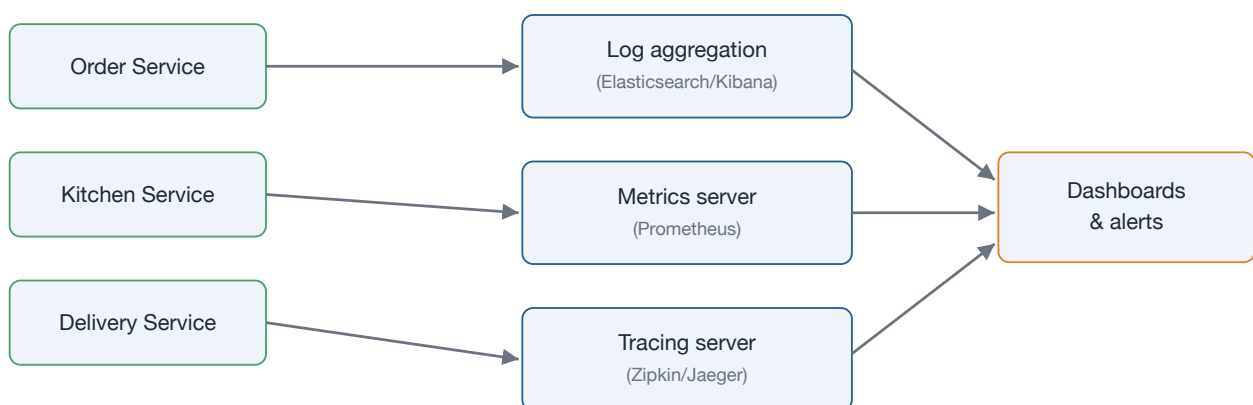
When to use: Any service that matters in production — it turns "noise" into "signal."

Pattern: Audit logging

Problem: You need a record of who did what — for security investigations, compliance, and customer support ("who cancelled this order?").

Solution: Log business-significant user actions (with user id, timestamp, action, affected entity) to a durable audit trail, separate from debug/application logs.

When to use: Anywhere users take actions with business or compliance consequences — payments, refunds, account changes.



Every service pushes logs, metrics, and trace spans to the same central stores.

Figure 11.3 — Observability stack: services feed one log store, one metrics store, one tracing store, which feed shared dashboards and alerts.

Microservice Chassis pattern

Pattern: **Microservice chassis**

Problem: Every one of FTGO's dozens of services would otherwise have to reimplement config loading, service discovery, health checks, metrics, tracing, and circuit breakers from scratch. That's slow to build and inconsistent across teams.

Solution: Build (or adopt) a base framework/library — a **chassis** — that every service starts from, which handles those cross-cutting concerns out of the box. Examples: Spring Boot + Spring Cloud, Netflix OSS's stack (Eureka, Hystrix, Ribbon), Dropwizard.

When to use: When your organization has many services in the same language/platform and wants consistency without every team hand-rolling the same plumbing.

Analogy: A chassis is like using a car platform shared across several models — the frame, engine mounts, and electronics harness are already there, so each model team only designs the body and interior. Every FTGO service built on the same chassis gets health checks and tracing "for free," instead of each team inventing their own.

Service Mesh: pushing it into the infrastructure

A chassis lives inside your application code, which means it's tied to a language/framework — a chassis built for Java Spring Boot doesn't help a Node.js or Go service. A **service mesh** (Istio, Linkerd) solves this by moving cross-cutting concerns out of the application entirely, into a network-level **sidecar proxy** deployed alongside each service instance. The sidecar handles retries, timeouts, circuit breaking, mTLS, metrics, and tracing headers — regardless of what language the service is written in.

TRADEOFFS / WATCH OUT

Chassis lock-in vs service-mesh operational cost. A chassis couples every service to one language/framework and one library's upgrade cadence — great consistency, but painful if a team wants to use a different stack, and a security patch means redeploying every service. A service mesh is language-agnostic and decouples app code from infra concerns, but adds a sidecar proxy per pod (extra latency, memory, and a whole new piece of infrastructure — the mesh control plane — that itself needs to be operated, upgraded, and debugged). Many staff engineers choose a lightweight chassis first, and only reach for a mesh once polyglot services or fine-grained traffic control (canary releases, mTLS everywhere) justify the operational overhead.

Staff-eng lens: When asked "chassis or mesh?" in a design review, the right answer is usually "it depends on your organization's stack diversity and platform team maturity" — not a blanket preference. A single-language shop with a strong platform team can get a long way with a shared chassis library. A polyglot org, or one that needs consistent security/observability enforced regardless of what any team builds, gets more value from a mesh, because the guarantees live in infrastructure, not in code any given team could accidentally skip.

Splitting FTGO into Order Service, Kitchen Service, Delivery Service and friends only helps if you can actually run all those services reliably. This chapter is about the "how do I ship this" problem: packaging, isolating, scaling, and rolling out dozens of small services without losing your mind. For a staff engineer, this is where architecture meets operational reality — the deployment choice you make shapes your team's on-call life for years.

The deployment options, from simplest to most automated

There are four broad ways to deploy a service. Each one trades simplicity for isolation and automation. As you move up the ladder, a crashing service is less likely to take its neighbors down with it, and rolling out changes gets safer — but you also take on more machinery to operate.

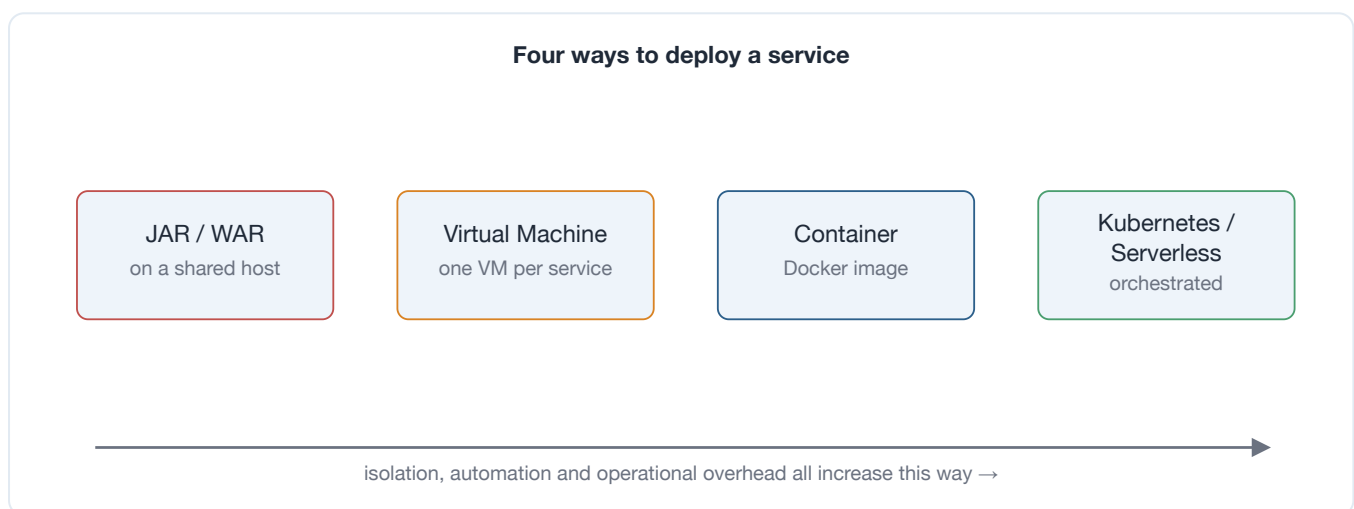


Figure 12.1 — Deployment options ladder: as isolation and automation go up, so does the machinery you must run.

1. Language-specific packaging format (JAR/WAR)

You build each service as a plain JAR or WAR file and run several of them on the same host or app server, like several Tomcat apps sharing one machine. It is the simplest option — no containers, no VMs, just "run the process."

TRADEOFFS / WATCH OUT

Weak isolation. If the Kitchen Service leaks memory or spins the CPU, it can starve the Order Service running right next to it. There's no clean way to say "this service gets 2 CPUs" — they all share the box. Also, mismatched dependencies between services can collide (one needs Java 8, another Java 17).

2. Deploy a service as a VM

Package the whole service, its runtime, and its OS into a machine image (an AMI on AWS, for example) and run one VM per service instance.

BENEFITS

Strong isolation — each service gets its own kernel, its own resource limits. A crash in one VM cannot directly starve another. Mature tooling (auto-scaling groups, VM images) already exists.

TRADEOFFS / WATCH OUT

Heavy and slow. Booting a VM can take minutes, and each VM carries a full OS copy, which wastes memory and disk. Building VM images is slower than building container images, which drags down your deploy pipeline.

3. Deploy a service as a container (Docker)

A container packages the service and its dependencies, but shares the host OS kernel — so it starts in seconds instead of minutes, and images are much smaller than VM images.

BENEFITS

Lightweight and fast to start. Good process- and filesystem-level isolation. Images are versioned and stored in a registry (like Docker Hub or a private ECR repo), so "deploy version 42 of Delivery Service" is a well-defined, repeatable action.

TRADEOFFS / WATCH OUT

Isolation is weaker than a VM (shared kernel), and by itself Docker doesn't handle placement, restart-on-crash, or scaling across a fleet of hosts — you still need something to schedule containers onto machines. That "something" is usually Kubernetes.

4. Kubernetes: container orchestration

Kubernetes takes a cluster of machines and a pile of container images, and figures out where to run everything, restarts what crashes, and can add more instances when load increases. The key objects you need to know:

- **Pod** — the smallest deployable unit; one or more tightly-coupled containers that always run together (e.g. Order Service container + a logging sidecar).
- **Deployment** — a declarative spec saying "keep N copies of this Pod running," and it handles rolling updates for you.
- **Service** — a stable network name/IP that load-balances across the Pods of a Deployment, even as Pods come and go.
- **Ingress** — the entry point that routes external HTTP traffic (e.g. [/orders](#)) to the right Service inside the cluster.

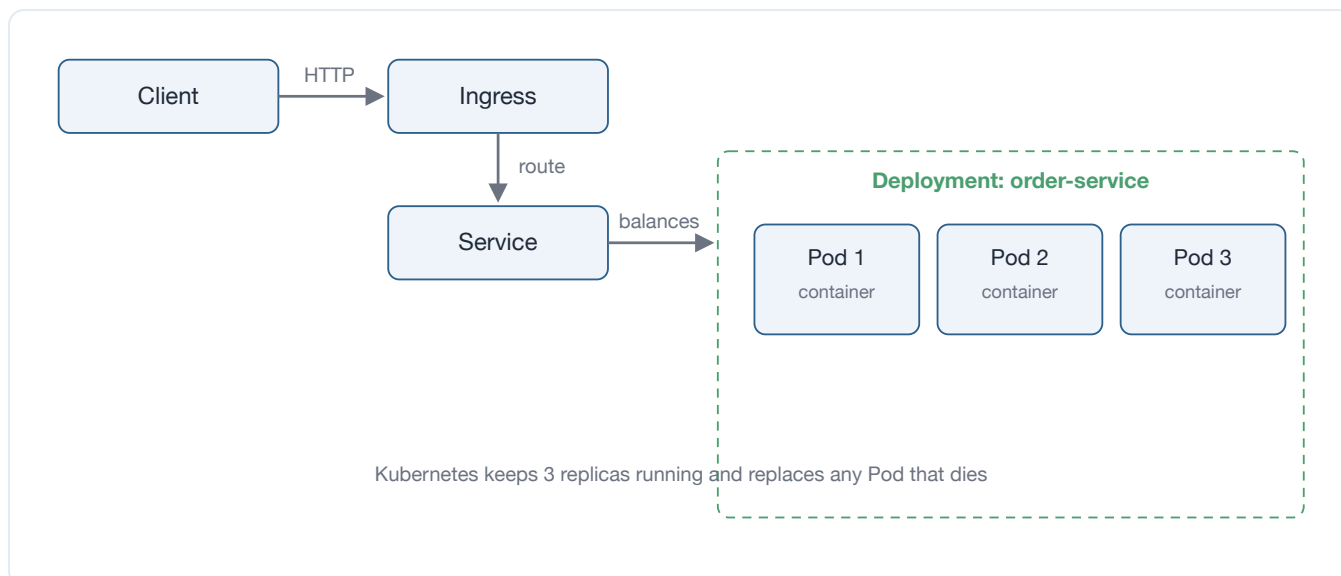


Figure 12.2 — Ingress routes to a Service, which load-balances across the Pods managed by a Deployment.

BENEFITS

Self-healing (dead Pods are replaced automatically), autoscaling (spin up more Pods under load), and rolling updates (replace old Pods with new ones gradually, with automatic rollback if health checks fail).

Staff-eng lens: Kubernetes buys you a lot, but it is not free. Someone has to run the cluster, manage upgrades, tune resource requests/limits, and debug YAML. For a five-person startup with three services, a well-run container setup on a PaaS may beat "let's run our own Kubernetes" — the ops tax outweighs the benefit until you have enough services and enough team to justify a platform team.

Service mesh

As you get more services, cross-cutting concerns pile up: retries, timeouts, circuit breakers, TLS between services, and tracing. A **service mesh** (like Istio) moves this logic out of application code and into a network layer, usually implemented as a **sidecar** proxy next to every service instance.

KEY IDEA

Every Pod gets a tiny proxy container (the sidecar) that intercepts all network traffic in and out. The mesh's control plane configures all those proxies centrally, so you get traffic shaping, mutual TLS, and observability without touching Order Service's or Kitchen Service's code.

The mesh is also what makes it practical to separate **deployment** from **release**: you can deploy a new version of Delivery Service alongside the old one, and use the mesh to send it 1% of traffic (a **canary release**), or flip all traffic at once from the old version to the new (**blue-green deployment**) and flip back instantly if something looks wrong.

Serverless deployment

With serverless (AWS Lambda is the classic example), you upload a function and a trigger (an HTTP call, a queue message, a schedule) and the platform runs it on demand — you never think about a server, a container,

or a Pod.

BENEFITS

No servers to patch or scale. You pay only for actual invocations. It scales to zero when idle and scales out automatically under load — handy for the Notification Service that fires occasionally but must survive a lunchtime spike.

TRADEOFFS / WATCH OUT

Cold starts — a function that hasn't run recently can take real time to spin up, which hurts latency-sensitive paths like checkout. Functions have time limits (e.g. 15 minutes on Lambda) and limited local resources, so long-running or heavy compute jobs don't fit. Debugging and local testing are harder than with a normal service.

Staff-eng lens: Serverless is great for the edges of your system — webhooks, glue code, scheduled jobs, event handlers. It's a much harder sell for the core Order Service that needs predictable low latency and long-lived connections. Match the deployment option to the shape of the workload, not to what's trendy.

Pattern: Language-specific packaging format

Problem: You need the simplest possible way to deploy a service.

Solution: Package the service using the language's native format (JAR, WAR, npm bundle) and run it directly on a host or app server, often sharing the host with other services.

When to use: Small teams, few services, or trusted internal tools where weak isolation is an acceptable risk.

Pattern: Deploy a service as a VM

Problem: You need strong isolation between services, and don't yet have container tooling.

Solution: Package the service, its runtime, and an OS into a machine image, and run each instance as its own VM.

When to use: Regulatory or security requirements demand hard isolation, or your infra is already VM-centric (e.g. classic AWS EC2 auto-scaling groups).

Pattern: Deploy a service as a container

Problem: VMs are too slow and heavy, but you still want good isolation and repeatable deploys.

Solution: Package the service into a container image (e.g. Docker) that shares the host kernel but has its own filesystem and process space; run it via an orchestrator.

When to use: The default choice for most microservices today, especially paired with Kubernetes.

Pattern: **Serverless deployment**

Problem: You don't want to manage any servers or containers, and load is spiky or intermittent.

Solution: Deploy the service as a function (e.g. AWS Lambda) that the platform runs on-demand in response to triggers, scaling automatically.

When to use: Event handlers, glue code, infrequent jobs — not for latency-critical, long-running, or resource-heavy work.

Pattern: **Service mesh**

Problem: Every service reimplements retries, timeouts, TLS, and tracing, and it's hard to control traffic centrally.

Solution: Add a network-level layer (control plane + per-instance sidecar proxies) that handles traffic management, security, and observability outside application code.

When to use: Enough services that cross-cutting network concerns are becoming a maintenance burden, and the team can absorb the extra operational complexity.

Pattern: **Sidecar**

Problem: You need to add infrastructure behavior (proxying, logging, config) to a service without modifying its code.

Solution: Deploy a helper container alongside the service in the same Pod, sharing its network and lifecycle, to handle the cross-cutting concern.

When to use: Service mesh proxies, log shippers, config watchers — anything that should live next to the service but not inside it.

You almost never get to build FTGO from a blank slate — usually there's already a monolith making money, and the job is to migrate it without breaking the business. This chapter is about doing that migration safely: growing microservices around the edges of the monolith instead of attempting a risky rewrite. For a staff engineer, this is often the actual job — most "microservices" work is migration, not greenfield design.

Why not just rewrite it?

A full rewrite ("big-bang") sounds appealing — throw out the old code, start clean. In practice it is one of the riskiest moves in software: you stop shipping features for months or years, the old system keeps changing under you, and you don't find out whether the new system actually works until the very end.

TRADEOFFS / WATCH OUT

Big-bang rewrites frequently run over budget, over time, and sometimes get cancelled entirely after the business loses patience. Meanwhile competitors keep shipping. The safer path is incremental: keep the monolith alive and useful while you carve pieces off it.

The Strangler Application (Strangler Fig)

Analogy: A strangler fig seed lands in the branches of a large tree and grows roots down around the trunk. Over years, the fig's roots thicken and wrap the host tree, until the original tree dies and rots away — but the fig, now self-supporting, keeps standing in exactly the same shape. Your microservices grow around the monolith the same way: the monolith keeps running while new services quietly take over piece by piece.

KEY IDEA

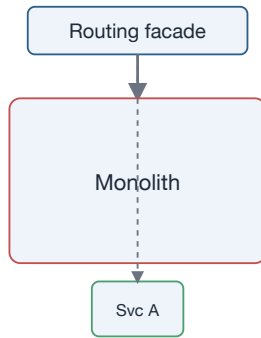
Put a routing facade in front of everything. Initially it sends all traffic to the monolith. Every time you extract a piece of functionality into a new service, you update the facade to route that traffic to the new service instead. Traffic to the monolith shrinks over time; nothing breaks in between.

The book describes three concrete strategies for applying this:

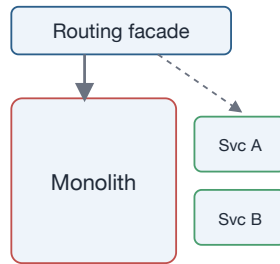
- **New features as services** — don't touch the monolith for new work; build every new feature (e.g. a new "loyalty points" feature) as a standalone service from day one.
- **Split frontend from backend** — separate the presentation layer from the business logic/data layer, so the UI can evolve independently and call services directly.
- **Extract functionality into services** — pick an existing module inside the monolith (e.g. Delivery scheduling) and pull it out into its own service, one module at a time.

Strangler application — three stages

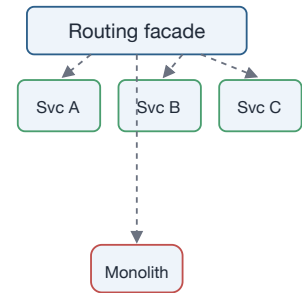
Stage 1: mostly monolith



Stage 2: mixed



Stage 3: mostly services



The facade's routing rules shift over time; clients never notice the difference

Figure 13.1 — Strangler application: the facade routes more and more traffic away from the shrinking monolith and toward new services.

EXAMPLE

FTGO could start by keeping Order Service inside the monolith but building the new Delivery scheduling optimizer as a standalone service. The API gateway (the facade) routes `/delivery/*` to the new service and everything else still goes to the monolith. Later, Order Service itself gets extracted the same way.

Staff-eng lens: The strangler approach is deliberately slow — that's the point. It trades speed for safety: the business keeps running the whole time, and you can stop or reverse course at any stage. The real bottleneck usually isn't technical, it's organizational — you need product buy-in to accept a multi-quarter migration with no flashy new features to show at first, and you need the team to resist the urge to "just rewrite this one part properly" outside the plan. Selling the roadmap is as much the job as writing the code.

Anti-Corruption Layer (ACL)

When you extract a new service, you want its internal domain model to be clean — new concepts, new names, no baggage. But it still needs to talk to the legacy monolith, whose model is often messy, inconsistent, or shaped by years of quick fixes. If you let the legacy model leak in directly, your shiny new service inherits the monolith's mess.

KEY IDEA

Put a translation layer between the new service and the monolith. It converts legacy concepts into the new service's clean model and back again, so neither side has to understand the other's internal shape.



ACL translates both ways; events keep data eventually consistent on each side

Figure 13.2 — The Anti-Corruption Layer translates between a new service's clean model and the monolith's legacy model.

During migration, the same piece of data (say, an order) often needs to exist on both sides for a while. The ACL, combined with events flowing in both directions, keeps the monolith's copy and the new service's copy consistent — the monolith publishes an event when it changes an order, the ACL translates and forwards it to the new service, and vice versa.

EXAMPLE

The monolith's **OrderEntity** has a dozen legacy fields, some unused, some named after concepts that no longer exist. The new Order Service defines a clean **Order** aggregate with just what it needs. The ACL is the only piece of code that knows about both shapes — it maps one to the other whenever an event crosses the boundary.

Staff-eng lens: The ACL is a deliberate, temporary layer of translation debt — you're accepting extra code and a bit of latency in exchange for keeping the new service's model clean while the old system still exists. Track it as migration debt with an explicit removal plan (once the monolith module is fully retired, delete the ACL), otherwise it quietly becomes permanent infrastructure nobody wants to own.

When not to use microservices

Microservices add real cost: network calls where there used to be method calls, distributed data consistency, more moving parts to deploy and monitor. If your system is small, your team is small, or the domain is still unclear, a well-structured monolith is often the right call. You can always split it later once you actually understand where the seams should be — splitting badly-guessed boundaries is more expensive than starting simple.

TRADEOFFS / WATCH OUT

Don't adopt microservices just because it's the industry trend. If a single small team can still hold the whole system in their heads and deploys aren't blocking each other, the operational tax of microservices (see Chapter 12) probably isn't worth paying yet.

Pattern: **Strangler application**

Problem: You need to migrate a monolith to microservices without a risky big-bang rewrite.

Solution: Put a routing facade in front of the system; build new functionality as services and incrementally extract existing functionality, shifting the facade's routing as each piece moves, until the monolith is gone or reduced to a core.

When to use: Any live system with real users where downtime and "big rewrite" risk are unacceptable — i.e. almost always.

Pattern: **Anti-corruption layer**

Problem: A new service needs to interact with a legacy system without importing its messy domain model.

Solution: Insert a translation layer between the new service and the legacy system that maps concepts both ways, keeping the new service's model clean and, via events, keeping data consistent across both sides during migration.

When to use: Whenever a service under active development must integrate with legacy code it does not want to be architecturally coupled to.



Cheat Sheet — Patterns at a Glance

A one-page map of the whole pattern language. In an interview, being able to name the problem each pattern solves — and its cost — is what separates senior from staff.

Group	Pattern	Solves	Main cost
Application arch.	Microservice architecture	Independent deploy & scaling of loosely-coupled services	Distributed-system complexity
Decomposition	By business capability / subdomain	Where to draw service boundaries	Getting boundaries wrong is expensive
Communication	RPI (REST/gRPC) & Messaging	Services talking to each other	Sync = coupling; async = complexity
Reliability	Circuit breaker	Stop cascading failures	Tuning thresholds; fallbacks
Discovery	Client-/server-side discovery	Finding service instances	Extra infra (registry / mesh)
Data consistency	Saga	Transactions across services without 2PC	No isolation; compensations; eventual consistency
Reliable messaging	Transactional outbox / log tailing	Atomically update DB and send message	Extra table / CDC pipeline
Business logic	Aggregate & domain event	Consistency boundaries; emit changes	Design discipline
Persistence	Event sourcing	Audit log + reliable event publishing	Query difficulty; learning curve
Querying	API composition / CQRS	Queries that span services	In-memory joins / replication lag
External API	API gateway / BFF	Single, client-shaped entry point	Can become bottleneck / SPOF
Testing	Consumer-driven contract tests	Verify interactions cheaply	Contract upkeep
Security	Access token (JWT)	Propagate identity across services	Token/key management
Cross-cutting	Externalized config & chassis	Don't reinvent config/observability	Framework lock-in
Observability	Tracing, metrics, log aggregation, health check	See inside a distributed system	Instrumentation effort
Deployment	Container / Kubernetes / serverless	Isolated, automated, scalable deploys	Operational complexity
Refactoring	Strangler + anti-corruption layer	Escape the monolith incrementally & safely	Slow; dual maintenance

THE ONE THING TO REMEMBER

Microservices are not free. They trade **development-time coupling** for **run-time complexity**. Adopt them to move faster with many teams — not because they are fashionable. Start with a well-structured monolith, and reach for microservices when scaling the *organization* and *deployment cadence* demands it. The patterns in this guide exist to pay down the run-time complexity you take on.

STAFF-ENGINEER INTERVIEW ANGLES

- **Boundaries:** justify service boundaries via business capabilities / bounded contexts, not tech layers.
- **Data:** explain why there are no distributed ACID transactions, and how sagas + outbox give you consistency without 2PC.
- **Queries:** know when to use API composition vs CQRS and the eventual-consistency cost.
- **Failure:** reason about partial failure, circuit breakers, timeouts, bulkheads, and availability math.
- **Observability:** correlation IDs, distributed tracing, and metrics are non-negotiable at scale.
- **Migration:** advocate incremental strangler migration over big-bang rewrites.